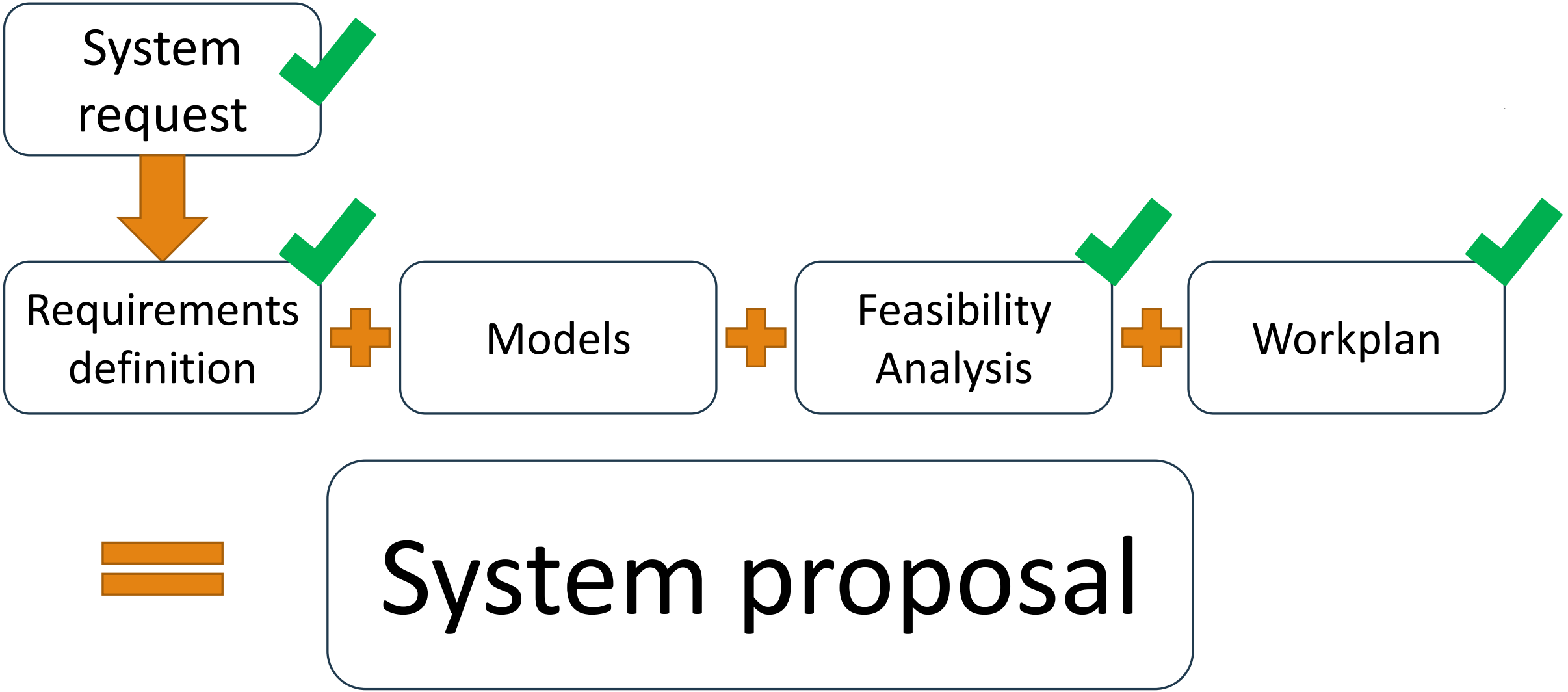
System Analysis

DR. DRAGUNOV

BUSINESS PROCESS and FUNCTIONAL MODELING

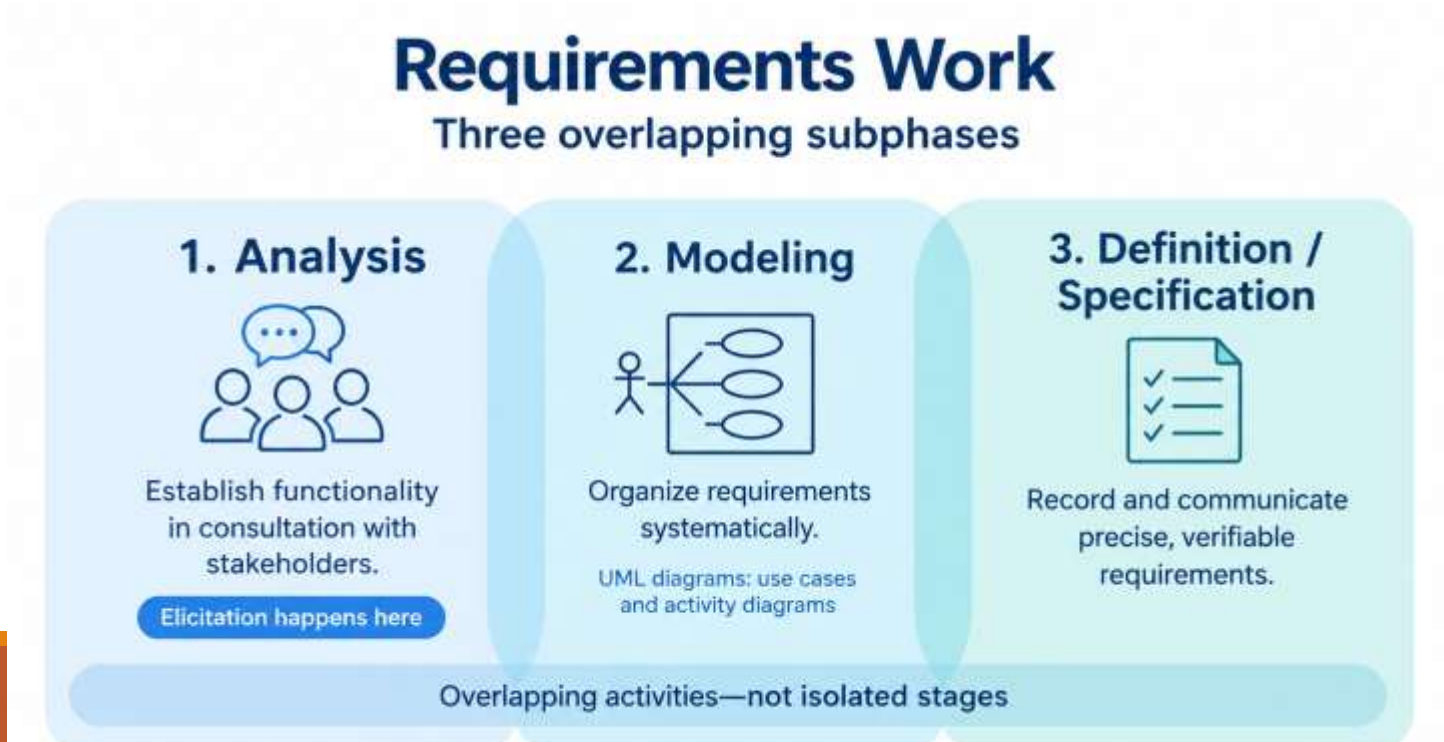
Analysis phase



The Three Subphases of Requirements Work

Requirements work proceeds through three overlapping subphases:

1. **Analysis** — Establish functionality in consultation with stakeholders. This is where elicitation happens.
2. **Modeling** — Organize requirements systematically, often using UML diagrams (use cases, activity diagrams).
3. **Definition/Specification** — Record and communicate precise, verifiable requirements.



Major Activities in Requirements Determination

Three major activities structure the determination process:

Requirements Anticipation. Predicting system characteristics based on previous experience—identifying likely problems, features, and requirements before deep investigation.

Requirements Investigation. Studying the current system and documenting its features using fact-finding techniques, prototyping, and computer-assisted tools. **This is the heart of systems analysis.**

Requirements Specification. Analyzing factual data, identifying essential requirements, describing features for the new system, and selecting strategies for fulfillment.

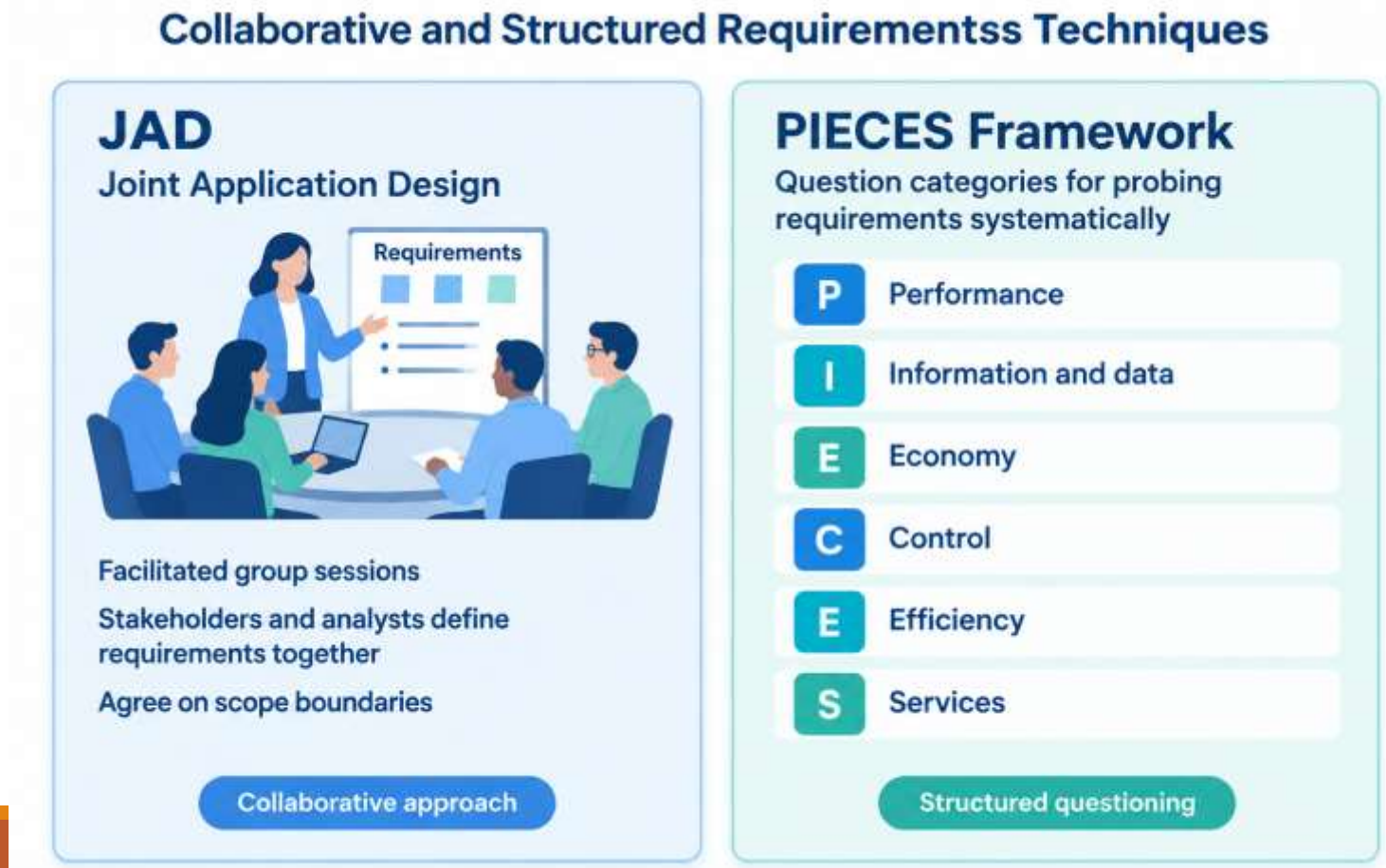


Fact-Finding (Elicitation) Techniques

Technique	How it works	Strengths	Limitations
Interview	Face-to-face with individuals or groups; structured (fixed questions) or unstructured (conversational)	Accurate, allows immediate clarification; unstructured can uncover in-depth, unanticipated information	Time-consuming; interviewer skill-dependent; may miss organizational breadth
Questionnaire	Series of logically ordered, clear questions distributed to many people	Cheap; reaches geographically dispersed stakeholders; good for usage patterns	No follow-up; low response rates; question design is difficult
Document analysis	Examining documents, reports, manuals, and publications about the system and organization	Non-intrusive; reveals formal rules and historical context	Documents may be outdated or incomplete
Observation	Analyst visits the organization and watches the actual workflow, document flow, and system usage	Captures what people do, not just what they say they do	Observer effect; time-intensive; may miss infrequent but critical events

JAD (Joint Application Design) (don’t forget)

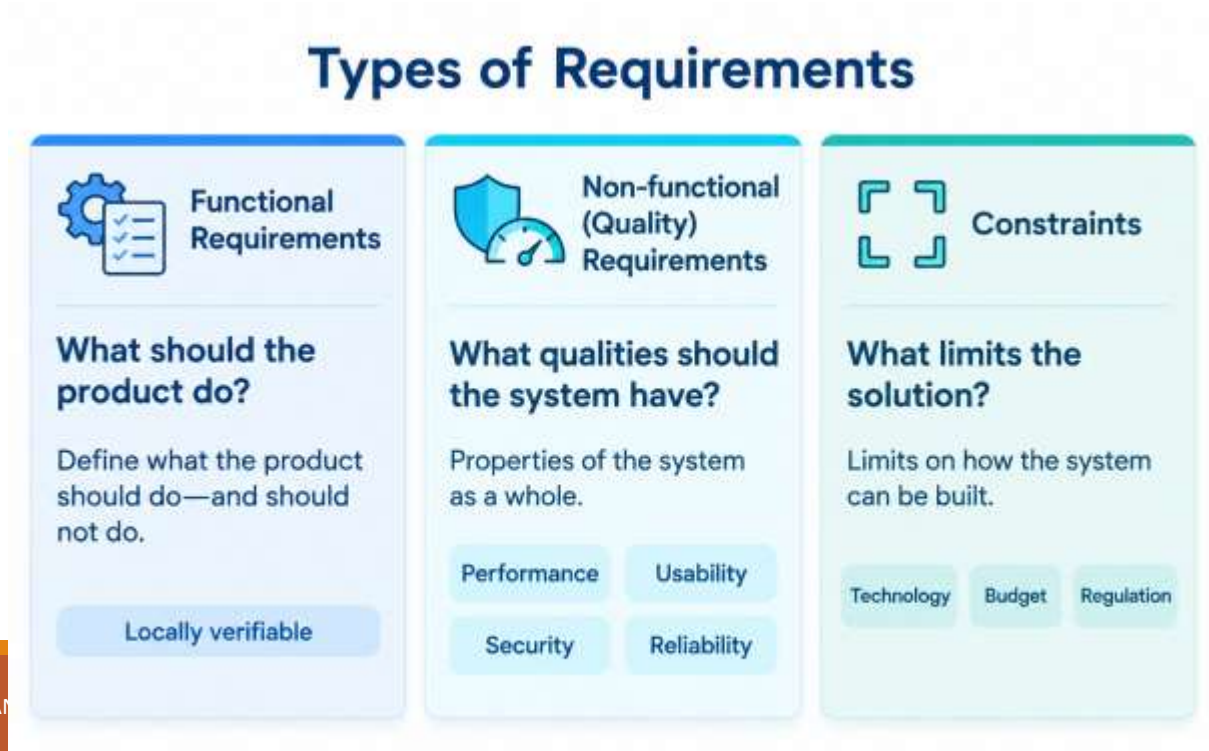
Collaborative and structured extensions include **JAD (Joint Application Design)** — facilitated group sessions where stakeholders and analysts define requirements and bound scope together — and the **PIECES framework** — a taxonomy of question categories (Performance, Information and data, Economy, Control, Efficiency, Services) used to probe requirements systematically.



Types of Requirements

- **Functional requirements** — what the product should do (and should not do); locally verifiable.
- **Non-functional (quality) requirements** — properties of the system as a whole (performance, usability, security, reliability).
- **Constraints** — limits on how the system can be built (technology, budget, regulation).

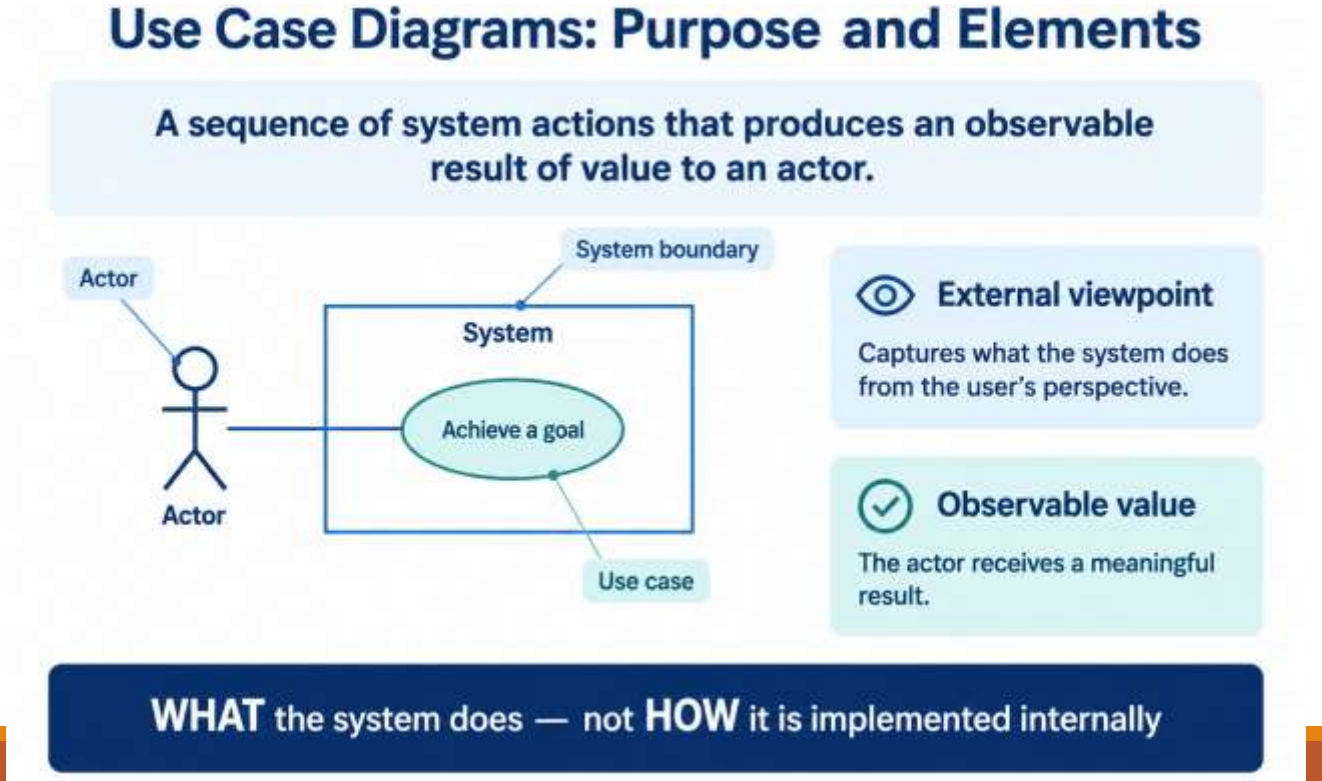
A well-formed requirement should be **verifiable**: for example, *"Visual feedback from tapping a control shall be displayed within 100 ms of the tap."*



UML Use Cases and Activity Diagrams (Functional modeling)

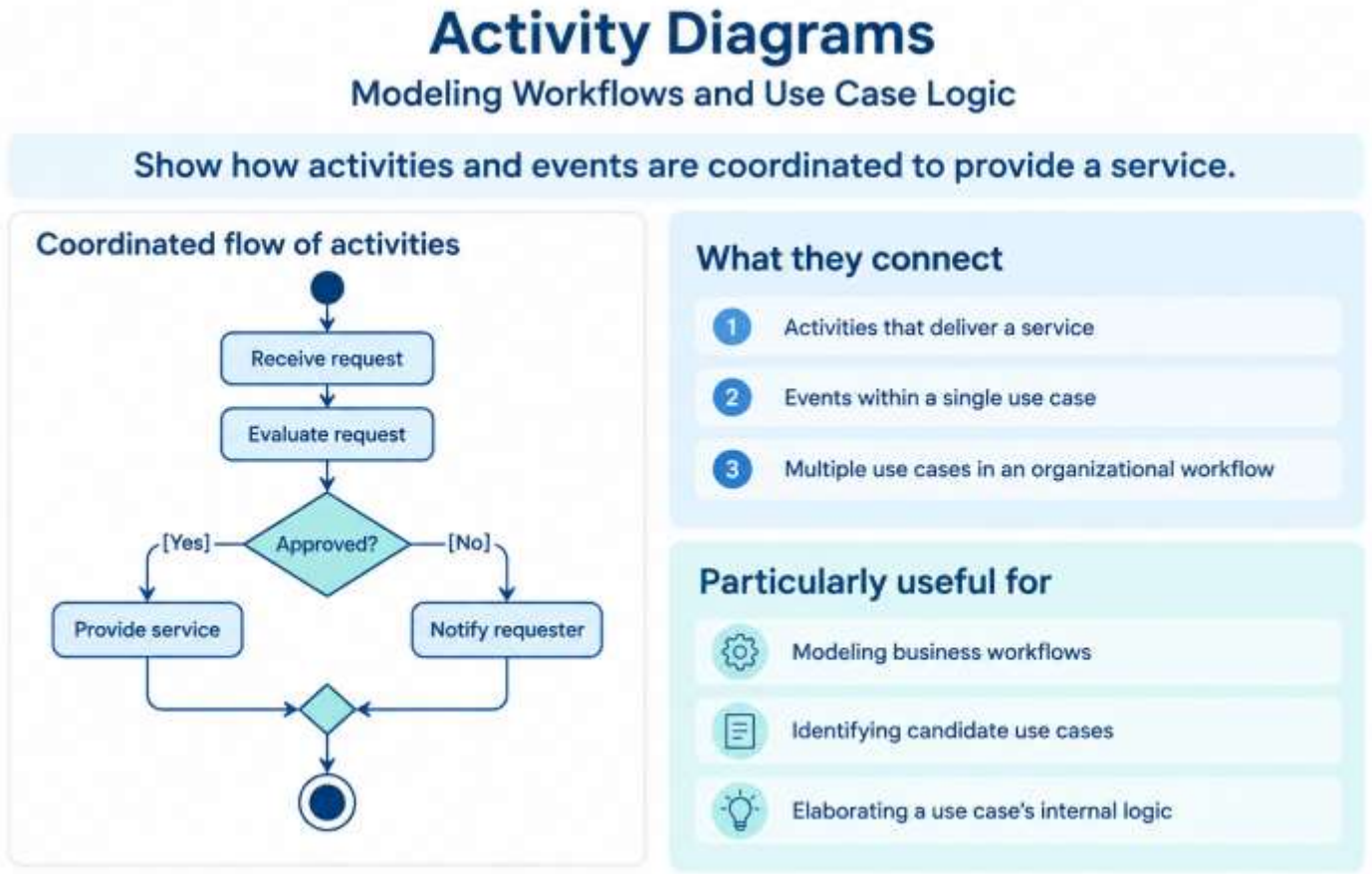
Use Case Diagrams: Purpose and Elements

A **use case** describes a sequence of actions a system performs that yields an observable result of value to a particular **actor**. Use cases model the system at its **external interface**—they capture *what* the system does from the user's viewpoint, not *how* it is implemented internally.



Activity Diagrams: Modeling Workflows and Use Case Logic

Activity diagrams describe how activities are coordinated to provide a service, how events within a single use case relate to one another, and how a collection of use cases coordinates into an organizational workflow. They are particularly useful for modeling business workflows, identifying candidate use cases, and elaborating the internal logic of a use case.



Using AI to Generate Use Case Diagrams from Natural Language

Manual UML modeling is **time-consuming and error-prone**, especially in iterative projects where requirements change frequently and use case diagrams accumulate ambiguities, redundancies, and inconsistencies. LLMs offer a promising shortcut: transform a natural-language requirements description directly into a structured diagram.

Recent empirical work suggests the gap between human and machine is narrowing. In a 2025 study evaluating LLM agents on 17 use case diagram modeling exercises designed for master's students, LLMs achieved **comparable completeness and redundancy** to human solutions, with no statistically significant difference. Specialized fine-tuned models (e.g., GLM4UML) report F1 scores around 72% on use case generation, outperforming several open-source LLMs and approaching proprietary models on some metrics.

From Requirements to UML with LLMs

Manual UML Modeling



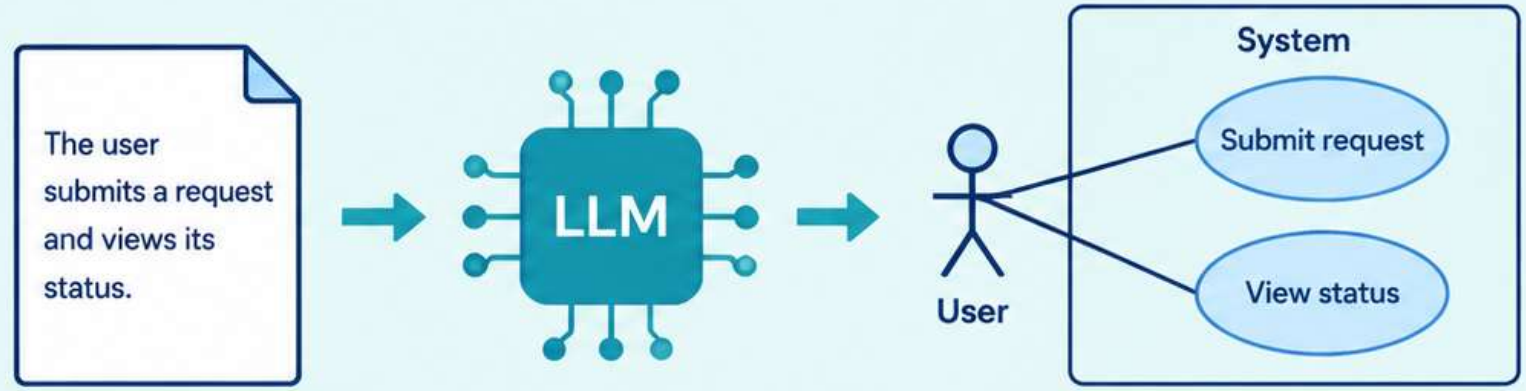
Time-consuming and error-prone

Frequent requirements changes

- Ambiguities
- Redundancies
- Inconsistencies

LLM-Assisted Modeling

A promising shortrcut



Natural-language requirements

Structured UML diagram

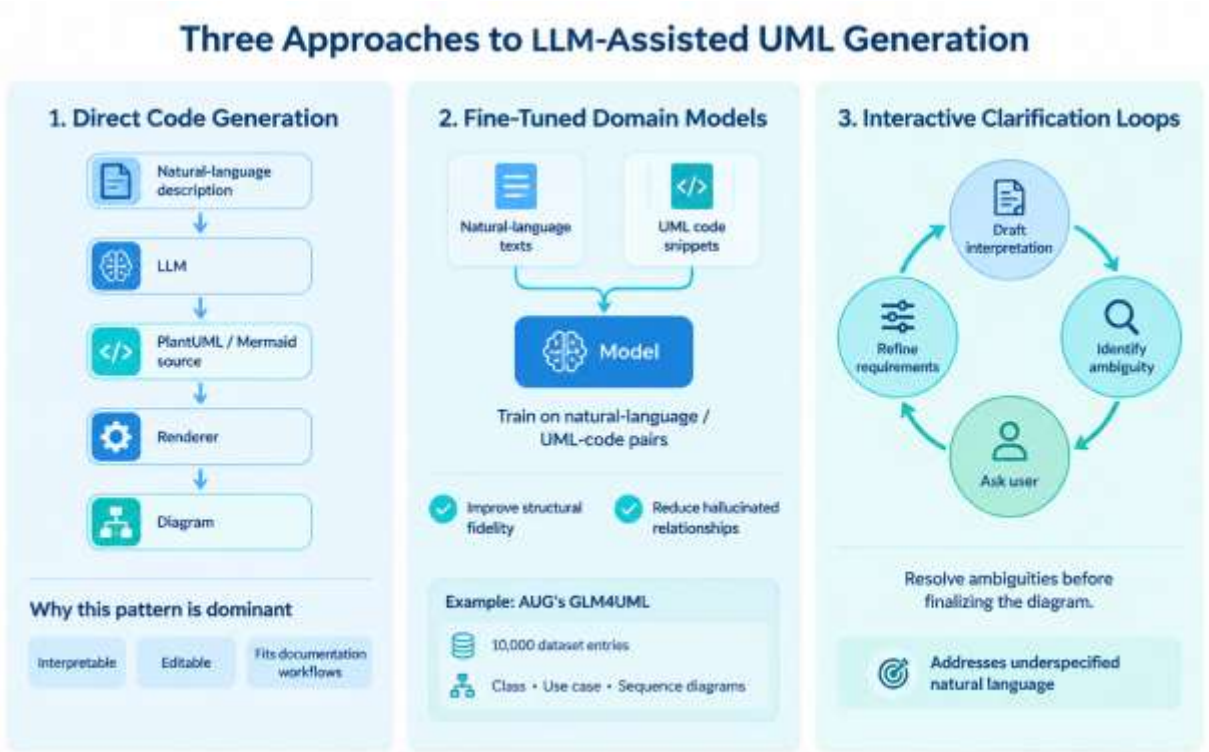
Transform a requirements description into a structured diagram

Architectural Approaches

1. **Direct code generation.** The LLM reads the natural-language description and emits **PlantUML** or **Mermaid** source, which a renderer converts to a diagram. This is the dominant pattern because it is interpretable, editable, and integrates with documentation workflows.

2. **Fine-tuned domain models.** A general-purpose LLM is fine-tuned on a dataset of (natural-language, UML-code) pairs to improve structural fidelity and reduce hallucinated relationships. AUG's GLM4UML is an example, built on 10,000 entries and supporting class, use case, and sequence diagrams.

3. **Interactive clarification loops.** Rather than one-shot generation, the tool identifies ambiguities and asks the user to resolve them before finalizing the diagram—addressing the core problem that natural language is inherently underspecified.



Tooling landscape.

Visual Paradigm's VPasCode converts natural-language descriptions into PlantUML or Mermaid diagram code with live preview and AI refinement. Open-source projects such as uml-gen use TypeScript and Mermaid.js to generate use case and activity diagrams from text. The key enabler is the diagram-as-code paradigm: the LLM's output is not a pixel image but editable, version-controllable source.

Visual Paradigm VPasCode: Comprehensive Guide

A Practical Prompting Workflow

A disciplined workflow reduces the most common failure modes (invented actors, missing «include»/«extend» relationships, and scope creep):

1. **Prepare the input.** Rewrite the raw description into a structured brief: list the **actors**, the **system boundary**, and the **goals** each actor pursues [/pərˈsuːz/ follow (someone or something)]. Ambiguity in the input becomes hallucination in the output.
2. **Specify the notation and constraints.** State the target format (PlantUML or Mermaid), the relationship stereotypes to consider («include», «extend», generalization), and explicit exclusions (*"Do not invent external systems not mentioned."*).
3. **Generate, then audit against the text.** For every actor and use case in the diagram, trace it back to a sentence in the description. Flag relationships that lack textual evidence. Check that mandatory shared behavior is modeled as «include» and optional behavior as «extend»—LLMs frequently confuse the two.
4. **Iterate with focused corrections.** Ask the model to revise specific relationships or merge redundant use cases rather than regenerating from scratch. Interactive tools support this refinement loop natively.
5. **Validate structurally.** Render the generated code and inspect it for syntax errors, unreachable use cases, actors with no associations, and missing system boundaries.

Summary Comparison

Aspect	Manual modeling	LLM-assisted modeling
Speed	Slow; hours per diagram	Seconds to minutes for a first draft
Consistency	Depends on modeler skill	Structurally consistent; may repeat systematic errors
Ambiguity handling	Analyst resolves through elicitation	One-shot models guess; interactive tools ask
Best use	Final specification, complex domains	Rapid prototyping, educational feedback, first drafts
Human role	Author	Auditor and corrector

In-Class Practice: Generating a Use Case Diagram with AI

HerbHub — Rooftop Greenhouse Irrigation

HerbHub manages irrigation for a community rooftop greenhouse with 20 numbered plots rented by local residents.

Plot holders can look up the soil moisture reading for their own plot and book a watering slot with the shared hose. A plot holder may book at most three watering slots per week. Plot holders can cancel a booking; if the cancellation happens less than 24 hours before the slot, a small fee is charged.

The garden coordinator approves new plot applications and can suspend a plot holder's account. The coordinator also runs a monthly water usage report for the building committee.

HerbHub talks to two outside systems. The **Weather Service** supplies a daily forecast, which HerbHub uses to skip watering when rain is expected. The **Irrigation Controller** opens and closes the valves: HerbHub sends it watering commands and receives fault alerts from it.

If the controller reports a faulty valve, the coordinator may manually open a valve, bypassing the schedule. Fault alerts are also logged for maintenance.

Prompt A — the naive prompt

Here is a description of a system. Draw a use case diagram.

[Paste description]

Prompt B — the constrained prompt

You are a requirements analyst producing a UML use case diagram.

TASK: Produce a PlantUML use case diagram from the description below.

NOTATION: actors outside a system boundary rectangle; use cases as verb-noun names; use <<include>> only for behavior that is mandatory every time; use <<extend>> only for optional/conditional behavior and label the condition; use generalization for actor roles that share use cases.

CONSTRAINTS:

- Use ONLY actors explicitly named or clearly implied in the text.
- Do not invent actors, systems, or features.
- Do not model business rules (limits, fees, quantities) as use cases.
- Mark each use case you are unsure about with a comment: '? <reason>'.

OUTPUT: (1) PlantUML source, (2) a traceability table mapping every actor and use case to the sentence that justifies it, (3) a list of ambiguities in the description and how you resolved them.

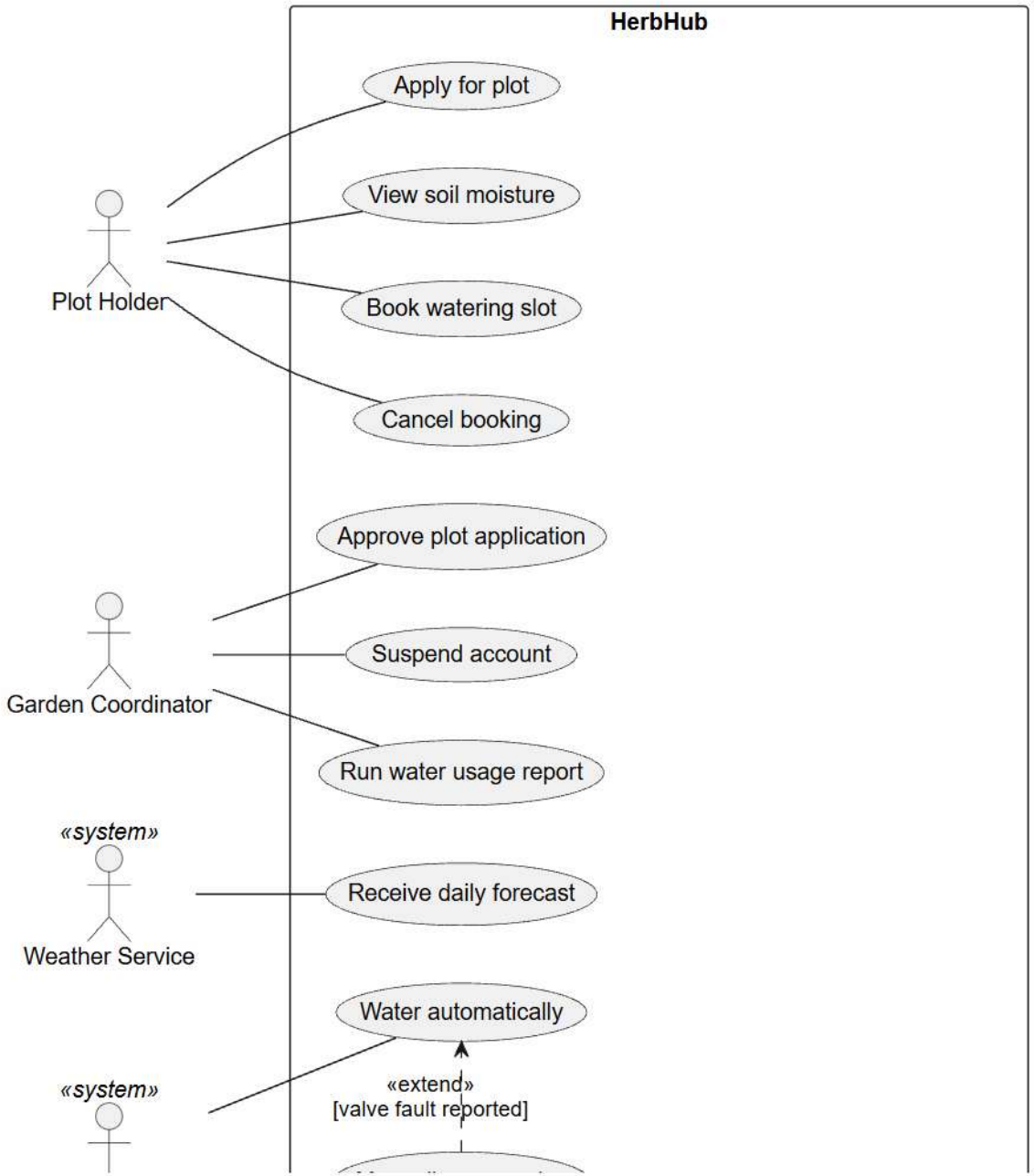
DESCRIPTION:

[Paste description]

```
@startuml
left to right direction
actor "Plot Holder" as PH
actor "Garden Coordinator" as GC
actor "Weather Service" as WS <<system>>
actor "Irrigation Controller" as IC <<system>>

rectangle HerbHub {
  usecase "Apply for plot" as U1
  usecase "Approve plot application" as U2
  usecase "Suspend account" as U3
  usecase "View soil moisture" as U4
  usecase "Book watering slot" as U5
  usecase "Cancel booking" as U6
  usecase "Run water usage report" as U7
  usecase "Receive daily forecast" as U8
  usecase "Water automatically" as U9
  usecase "Receive fault alert" as U10
  usecase "Log fault alert" as U11
  usecase "Manually open valve" as U12
}

PH -- U1
PH -- U4
PH -- U5
PH -- U6
GC -- U2
GC -- U3
GC -- U7
WS -- U8
IC -- U9
```



Audit against the text

Render Prompt B's output and complete this table for **every** model element:

Model element	**Justifying sentence (or blank)**	**Verdict**
e.g., Book watering slot	"Plot holders can … book a watering slot"	keep
e.g., Pay fee	(blank)	**delete**

Then check the classic AI error list:

- ☐ Invented actors (a Building Committee, an Administrator, a Payment Gateway)
- ☐ Business rules promoted to use cases ("at most three slots per week", "fee if < 24 h")
- ☐ «Extend» arrow pointing the wrong way (it must point from the optional behavior to the base)
- ☐ «Extend» used where the behavior is mandatory
- ☐ Non-human actors (Weather Service, Irrigation Controller) missing or drawn as human
- ☐ Generic names: Manage system, Handle data, Process request
- ☐ Two near-identical use cases instead of an actor generalization
- ☐ No system boundary

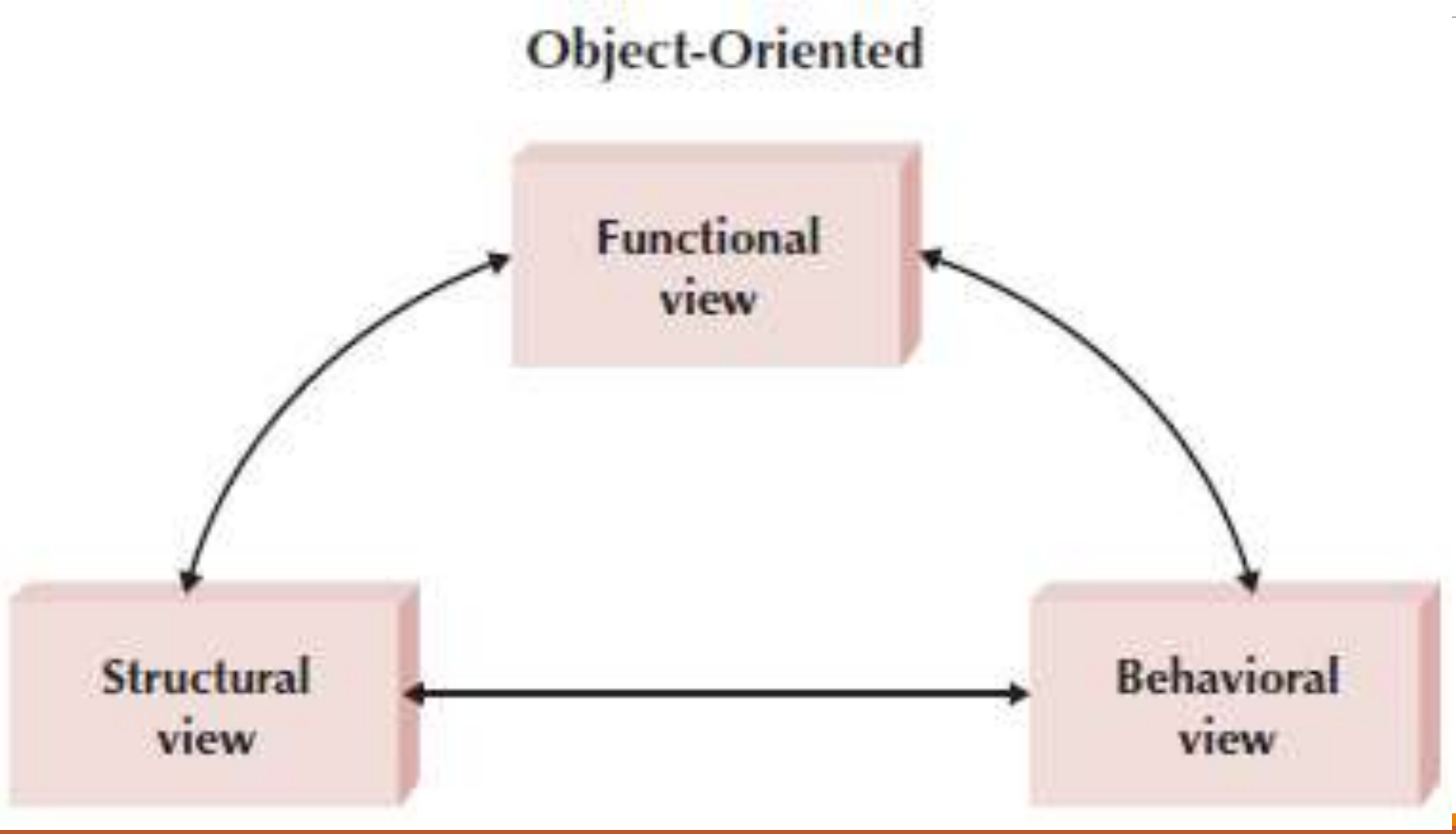
Iterate

Do not regenerate from scratch. Send targeted corrections:

Keep the diagram otherwise unchanged. Corrections:

1. Remove actor <X> — not in the text.
2. <Business rule> is a rule, not a use case: delete use case <Y>.
3. Reverse the <<extend>> arrow between <A> and <B> and label the condition.
4. Add the non-human actor <Z> with associations to <...>.
5. Re-output only: the full revised PlantUML and an updated traceability table.

All three architectural views of the system are interlinked and dependent on each other



BUSINESS PROCESS AND FUNCTIONAL MODELING

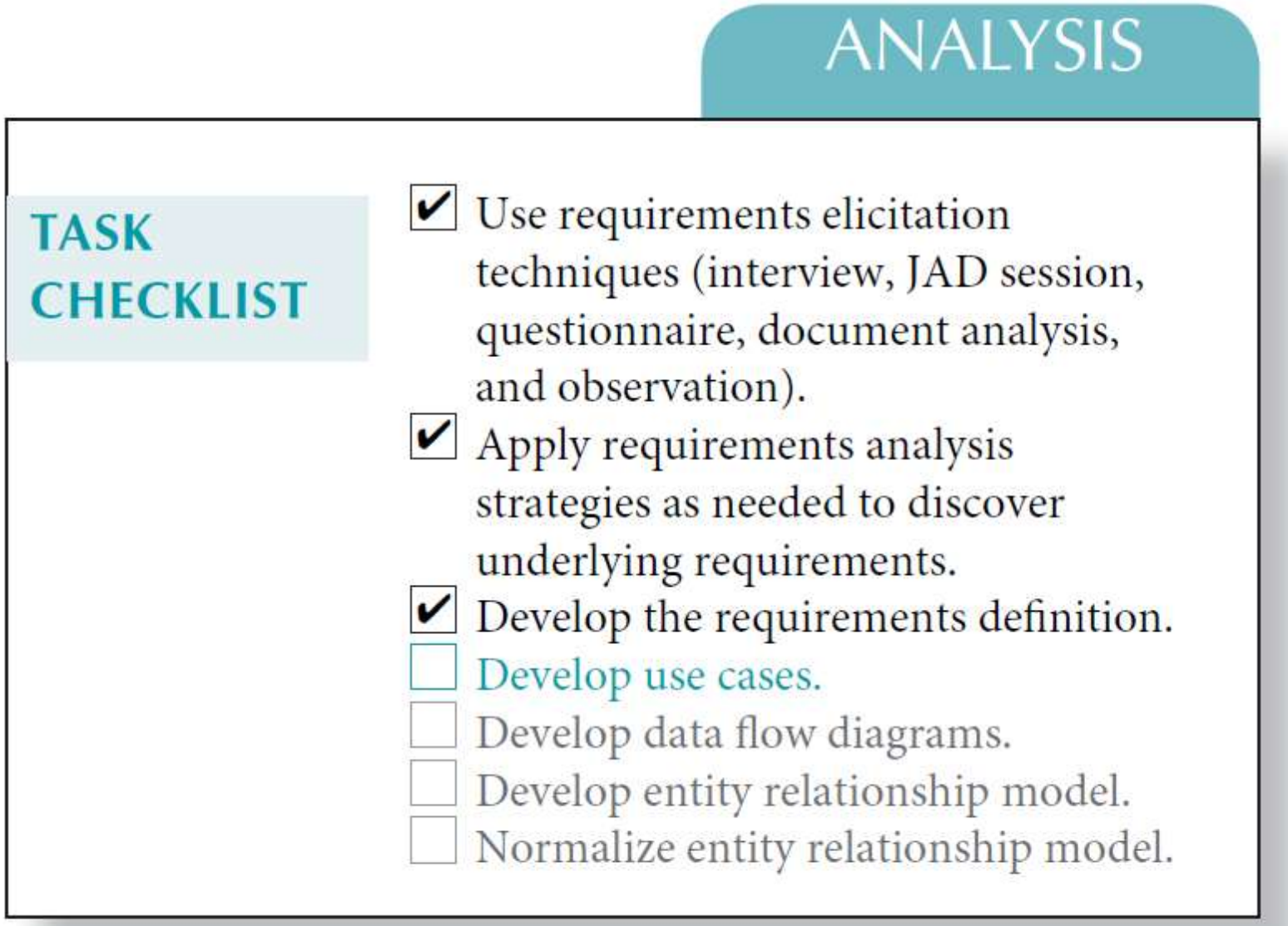
Functional models describe business processes and the interaction of an information system with its environment.

In object-oriented systems development, two types of models are used to describe the functionality of an information system:

- use cases diagrams
- activity diagrams.

Use cases are used to describe the basic functions of the information system. Activity diagrams support the logical modeling of business processes and workflows. Both can be used to describe the current as-is system and the to-be system being developed. We'll describe business process and functional modeling as a means to document and understand requirements and to understand the functional or external behavior of the system.

USE CASE ANALYSIS



WHAT IS A USE CASE?

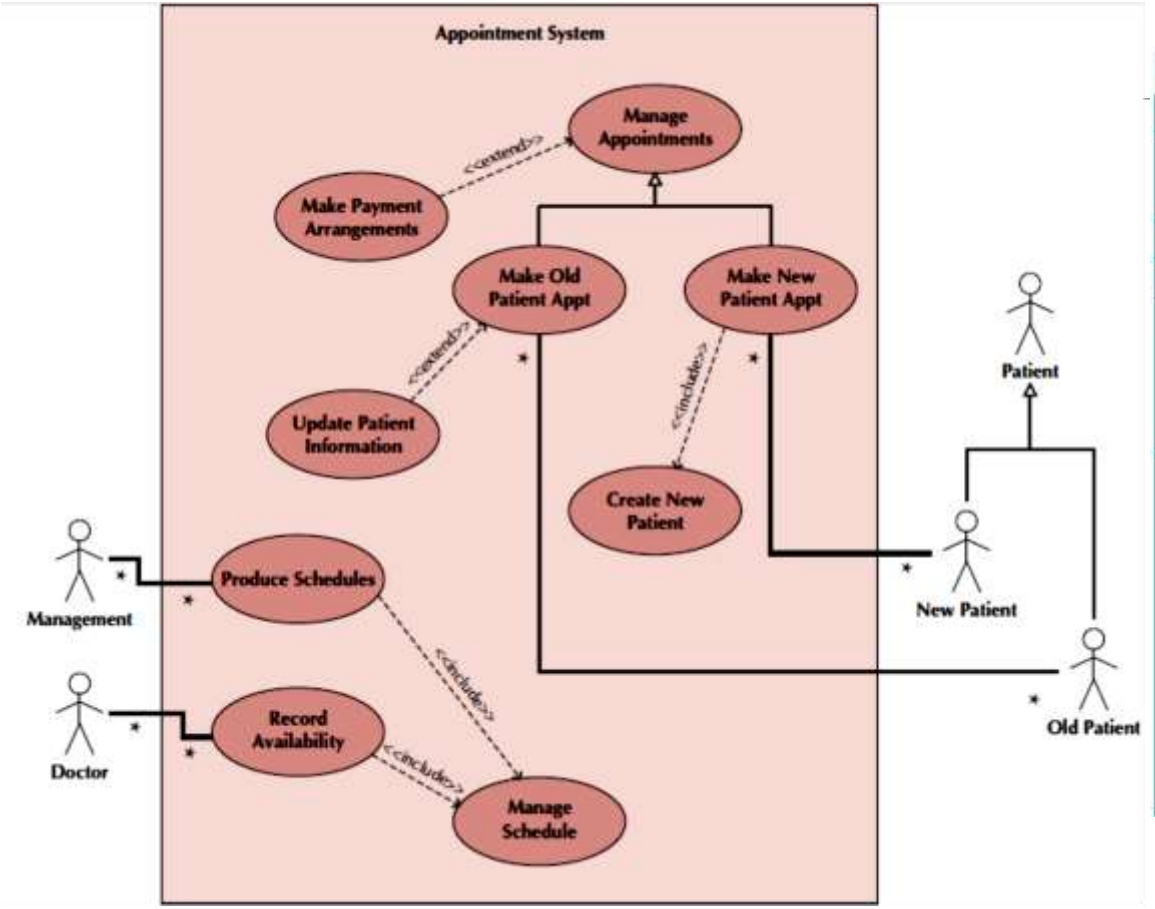
A use case depicts a set of activities performed to produce some output result. Each use case describes how an event triggers actions performed by the system and the user. With this type of event-driven modeling , everything in the system can be thought of as a response to some trigger event. When there are no events, the system is at rest, patiently waiting for the next event to trigger it. When a trigger event occurs, the system (and the people using it) responds, performs the actions defined in the use case, and then returns to the waiting state.

Use case

A *use case* is a formal way of representing the way a business system interacts with its environment.

It illustrates the activities performed by the users of the system. Use case modeling is often thought of as an external or functional view of a business process in that it shows how the users view the process rather than the internal mechanisms by which the process and supporting systems operate. Use cases can document the current system (i.e., as-is system) or the new system being developed (i.e., to-be system).

USE CASES



Use Case Name: Request a chemical	**ID:** UC-2	**Priority:** High
Actor: Lawn Chemical Applicator (LCA)		
Description: The Lawn Chemical Applicator (LCA) specifies the lawn chemical needed for a job by entering its name or ID number. The system satisfies the request by reserving the quantity requested or the quantity available and notifying the Chemical Supply Warehouse of the pick-up.		
Trigger: A Lawn Chemical Applicator (LCA) needs a chemical for a job.		
Type: ☑ External ☐ Temporal		
Preconditions: 1. The LCA identity is authenticated. 2. The LCA has necessary training and credentials on file. 3. The Chemical Supply datastore is up-to-date and on-line.		
Normal Course: 1.0 Request a lawn chemical from the chemical supply warehouse. 1. The LCA specifies a chemical needed and the quantity needed 2. The system lists chemical and quantity on hand from Chemical Supply datastore a. If the quantity on hand is less than the quantity needed, the LCA specifies the quantity he will take b. Purchasing is notified of chemical shortage 3. The system gives the LCA a Chemical Pick-up Authorization for the quantity requested 4. The system notifies the Chemical Supply Warehouse of the chemical pick-up 5. The system stores the Lawn Chemical Request in the Chemical Request datastore		

Use Case Name: Request a chemical	**ID:** UC-2	**Priority:** High

Actor: Lawn Chemical Applicator (LCA)

Description: The Lawn Chemical Applicator (LCA) specifies the lawn chemical needed for a job by entering its name or ID number. The system satisfies the request by reserving the quantity requested or the quantity available and notifying the Chemical Supply Warehouse of the pick-up.

Trigger: A Lawn Chemical Applicator (LCA) needs a chemical for a job.

Type: ☑ External ☐ Temporal

Preconditions:

1. The LCA identity is authenticated.
2. The LCA has necessary training and credentials on file.
3. The Chemical Supply datastore is up-to-date and on-line.

Normal Course:

1.0 Request a lawn chemical from the chemical supply warehouse.

1. The LCA specifies a chemical needed and the quantity needed
2. The system lists chemical and quantity on hand from Chemical Supply datastore
   a. If the quantity on hand is less than the quantity needed, the LCA specifies the quantity he will take
   b. Purchasing is notified of chemical shortage
3. The system gives the LCA a Chemical Pick-up Authorization for the quantity requested
4. The system notifies the Chemical Supply Warehouse of the chemical pick-up
5. The system stores the Lawn Chemical Request in the Chemical Request datastore

Postconditions:

1. The Lawn Chemical Request is stored in the Chemical Management System.
2. The Chemical Pick-up Authorization is produced for the LCA.
3. The Chemical Supply Warehouse is notified of the chemical pick-up.
4. Purchasing is notified of chemical outage.

Exceptions:

E1: Chemical is no longer approved for use (occurs at step 1)

1. The system displays message "That chemical is no longer approved for use"
2. The system asks the LCA if he wants to request another chemical or to exit

3a. The LCA asks to request another chemical

4a. The system starts Normal Course again

3b. The LCA asks to exit

4b. The system terminates the use case

Basic (overview) Information

The overview information identifies the use case and provides basic background information about the use case. The ***use-case name*** should be a verb–noun phrase (e.g., Request a chemical). The ***use-case ID number*** provides a unique way to find every use case and also enables the team to trace design decisions back to a specific requirement.

The ***primary actor*** is usually the trigger of the use case—the person or thing that starts the execution of the use case. The **primary purpose of the use case** is to meet the goal of the primary actor. The ***brief description*** is typically a single sentence that describes the essence of the use case.

Importance level

The ***importance level*** can be used to prioritize the use cases.

The importance level enables the users to explicitly prioritize which business functions are most important and need to be part of the first version of the system and which are less important and can wait until later versions if necessary.

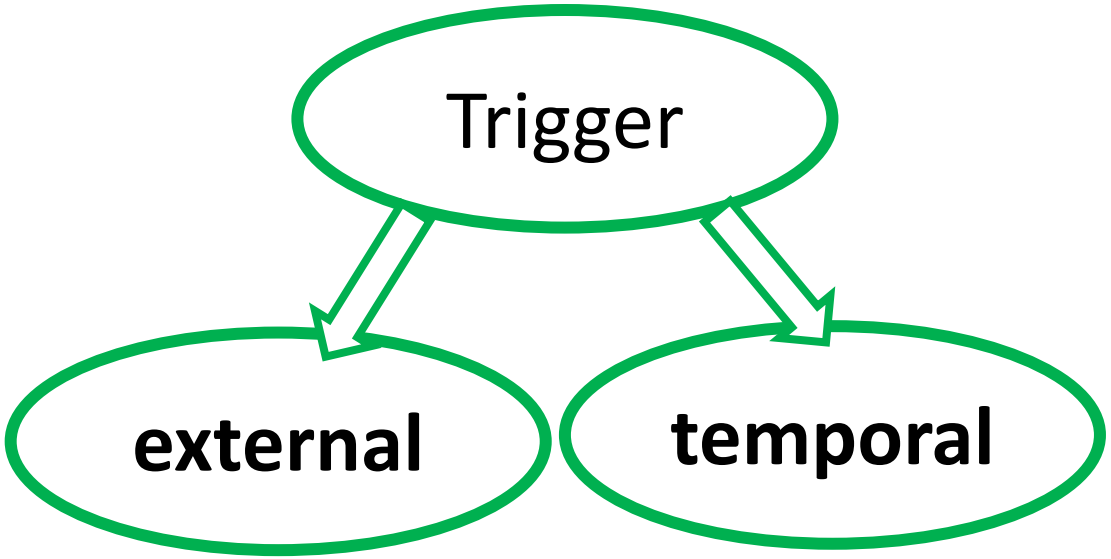
The importance level can use a fuzzy scale, such as high, medium, and low It can also be done more formally using a weighted average of a set of criteria.

For example, Larman suggests rating each use case over the following criteria using a scale from zero to five:

- The use case represents an important business process.
- The use case supports revenue generation or cost reduction.
- Technology needed to support the use case is new or risky and therefore requires considerable research.
- Functionality described in the use case is complex, risky, and/or time critical. Depending on a use case's complexity, it may be useful to consider splitting its implementation over several different versions.
- The use case could increase understanding of the evolving design relative to the effort expended.

A use case may have multiple *stakeholders* that have an interest in the use case. Each use case lists each of the stakeholders with each one's interest in the use case. The stakeholders' list always includes the primary actor.

Each use case typically has a *trigger* —the event that causes the use case to begin (e.g., Old Patient calls and asks for a new appointment or asks to cancel or change an existing appointment). A trigger can be an *external trigger*, such as a customer placing an order or the fire alarm ringing, or it can be a *temporal trigger*, such as a book being overdue at the library or the need to pay the rent.



Relationships some times also may be included

Use-case relationships explain how the use case is related to other use cases and users. There are four basic types of *relationships:*

- association
- extend
- include
- generalization.

Preconditions

Use cases are often performed in a sequence in order to accomplish an overall business task. When this practice is followed, it is important to define clearly what needs to be accomplished before each use case begins. These preconditions define the state the system must be in before the use case commences. In our example, you can see that in order for an LCA to request a chemical, he must be authenticated, his training and credentials must be up to date, and the datastore (a generic data repository) containing Chemical Supply information must be available and up to date. Th ese tasks are taken care of in a different use case prior to the performance of this use case. Once these preconditions are established, the LCA can perform the Request a chemical use case.

Normal Course

The next major part of a use case is the description of the major steps that are performed to execute the response to the event, the inputs used for the steps, and the outputs produced by the steps. The normal course lists the steps that are performed when everything flows smoothly in the system. This is sometimes called the “happy path ” because there are no problems or issues that arise when the steps are able to be followed normally.

Postconditions

In this section of the use case, we define the final products of this use case. In our example, the Lawn Chemical Request is stored, the LCA has the Chemical Pick-up Authorization, the Chemical Supply Warehouse is notified of the pick-up, and Purchasing is notified of any chemical outage. These postconditions also serve to define the preconditions for the next use case in the series. In our example, that would be the use case that describes the chemical pick-up at the Chemical Supply Warehouse.

Exceptions

In order to be complete, a use case should describe any error conditions or exceptions that may occur as the use case steps are performed. These are not normal branches in decision logic, but are unusual occurrences or errors that could potentially be encountered and will lead to an unsuccessful result. As the use case is written and reviewed, the analyst should ask the user if there are any special situations or errors that could occur with each step. If there are, they should be explained as an exception. We want to be sure that the system does not fail while in use because of an error that no one thought about. As you probably know, in many systems, handling exceptions can require more coding effort than the normal and alternative courses.

Fully Dressed Use Case Format

Use Case Name: Request a chemical	**ID:** UC-2	**Priority:** High

Actor: Lawn Chemical Applicator (LCA)

Description: The Lawn Chemical Applicator (LCA) specifies the lawn chemical needed for a job by entering its name or ID number. The system satisfies the request by reserving the quantity requested or the quantity available and notifying the Chemical Supply Warehouse of the pick-up.

Trigger: A Lawn Chemical Applicator (LCA) needs a chemical for a job.

Type: ☑ **External** ☐ **Temporal**

Preconditions:

1. The LCA identity is authenticated.
2. The LCA has necessary training and credentials on file.
3. The Chemical Supply datastore is up-to-date and on-line.

Normal Course:		**Information for Steps:**
1.0 Request a lawn chemical from the chemical supply warehouse.		
1. The LCA specifies the desired lawn chemical	←	Chemical name or ID
2. The system verifies the chemical is approved for usage	←	List of approved chemicals
3. The system displays the quantity of the lawn chemical on hand	←	Quantity on hand
4. The LCA specifies the quantity needed	←	Quantity needed
5. The system asks the LCA to confirm the request for the quantity needed or the quantity available (Alternative Course 1.1)	←	Request confirmation
6. The system gives the LCA a Chemical Pick-up Authorization for the quantity requested	→	Chemical Pick-up Authorization
7. The system notifies the Chemical Supply Warehouse of the chemical pick-up	→	Chemical Pick-up Notice
8. The system stores the Lawn Chemical Request in the Chemical Request datastore	→	Lawn Chemical Request

Alternative Courses

In this section, the steps followed for alternative paths through the use case are outlined. Alternative courses are included to depict branches in logic that also will lead to a successful conclusion of the use case. Notice that the location where the branch in logic from the normal course occurred is clearly stated. The course described in our example also depicts two potential paths through these steps. If the user decides to accept the quantity of chemical available, steps 2a–7a will be performed; however, if the user decides to cancel the request, steps 2b and 3b will be performed.

Alternative Courses:

1.1 Quantity available is less than quantity needed (branch at step 5)

1.	The system asks the LCA if he wants the quantity available or to cancel the request	
2a.	The LCA asks to take the quantity available	← Request quantity available
3a.	The system changes the quantity requested to the quantity available	
4a.	The system gives the LCA a Chemical Pick-up-Authorization for the quantity available	→ Chemical Pick-up Authorization
5a.	The system notifies the Chemical Supply Warehouse of the chemical pick-up	→ Chemical Pick-up Notice
6a.	The system stores the Lawn Chemical Request in the Chemical Management System	→ Lawn Chemical Request
7a.	The system notifies Purchasing of the chemical outage	→ Chemical Outage Notice
2b.	The LCA asks to cancel the request	← Cancellation
3b.	The system terminates the use case	

Postconditions:

1. The Lawn Chemical Request is stored in the Chemical Management System.
2. The Chemical Pick-up Authorization is produced for the LCA.
3. The Chemical Supply Warehouse is notified of the chemical pick-up.
4. Purchasing is notified of chemical outage.

Exceptions:

E1: Chemical is no longer approved for use (occurs at step 2)

1.	The system displays message "That chemical is no longer approved for use"
2.	The system asks the LCA if he wants to request another chemical or to exit
3a.	The LCA asks to request another chemical
4a.	The system starts Normal Course again
3b.	The LCA asks to exit
4b.	The system terminates the use case

Summary Inputs and Outputs

The final section of the use case summarizes the set of major inputs and outputs to the steps of the use case. Each of the major inputs and outputs to the use case are listed, along with its source or destination. These are all possible inputs and outputs, not just those that are part of the normal course. In this area, it is easy to see the inputs supplied by the LCA and the Chemical Supply datastore as the use case is performed, and the outputs produced and where the outputs go.

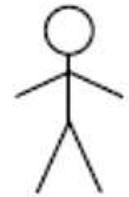
Summary Inputs	Source	Outputs	Destination
Chemical name or ID	LCA	Chemical Pick-up Authorization	LCA
List of approved chemicals	Lawn Chemicals Supply datastore		
Chemical quantity on hand	Lawn Chemicals Supply datastore	Chemical Pick-up Notice	Chemical Supply Warehouse
Quantity needed	LCA		
Request confirmation	LCA	Lawn Chemical Request	Chemical Request datastore
Request quantity available or cancellation	LCA	Chemical Outage Notice	Purchasing

BUSINESS PROCESS IDENTIFICATION WITH USE CASES AND USE-CASE DIAGRAMS

Elements of Use Case Diagrams

An actor:

- Is a person or system that derives benefit from and is external to the subject.
- Is depicted as either a stick figure (default) or, if a nonhuman actor is involved, as a rectangle with <<actor>> in it (alternative).
- Is labeled with its role.
- Can be associated with other actors using a specialization/superclass association, denoted by an arrow with a hollow arrowhead.
- Is placed outside the subject boundary.



Actor/Role

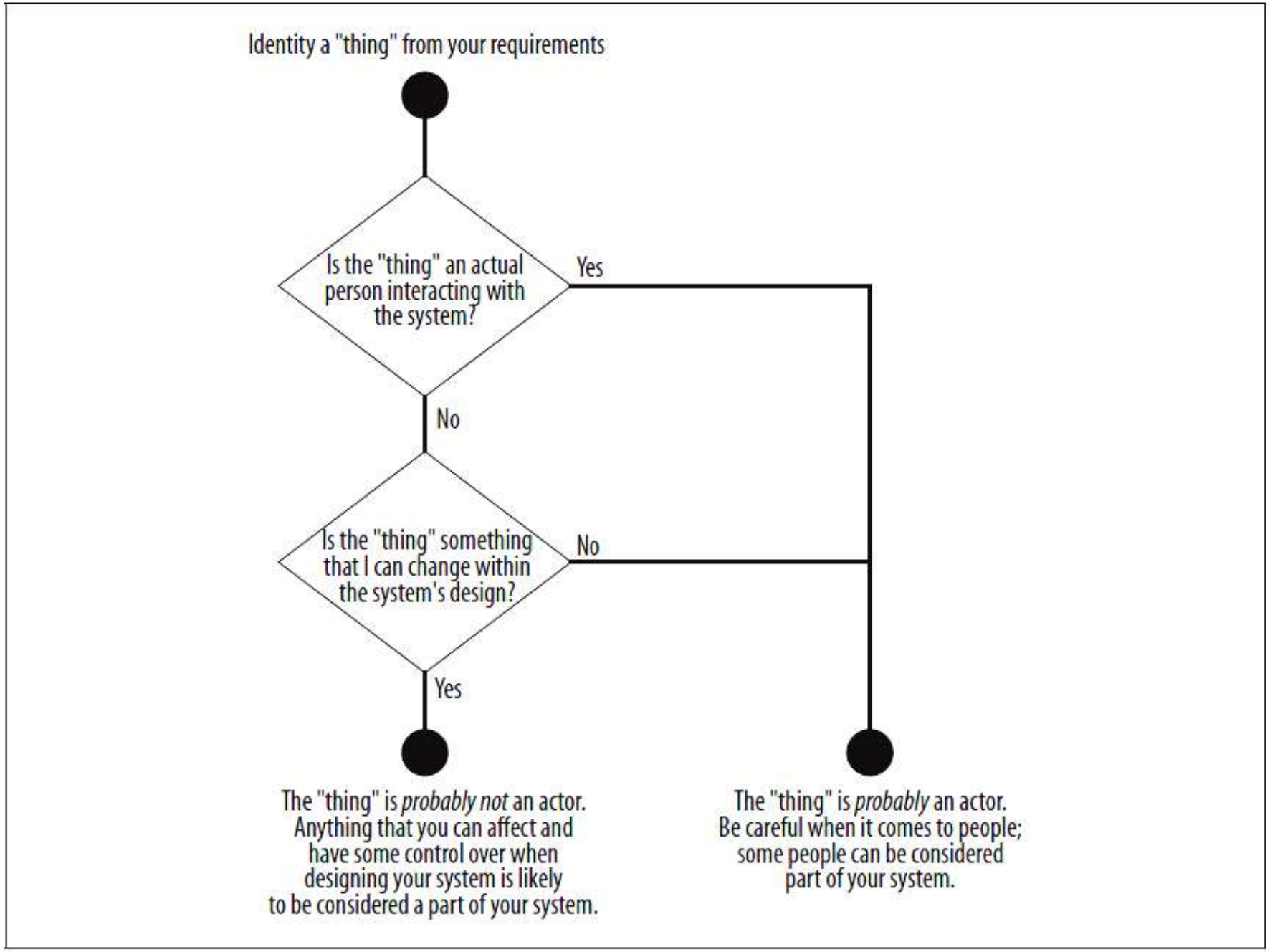
<<actor>>
Actor/Role

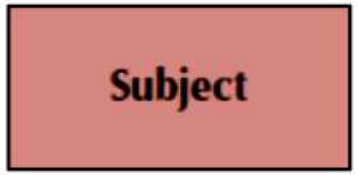
Actors

The stick figures on the diagram represent actors. An *actor* is not a specific user but instead is a role that a user can play while interacting with the system.

An actor can also represent another system in which the current system interacts. In this case, the actor optionally can be represented by a rectangle containing <<actor>> and the name of the system. Basically, actors represent the principal elements in the environment in which the system operates. Actors can provide input to the system, receive output from the system, or both.

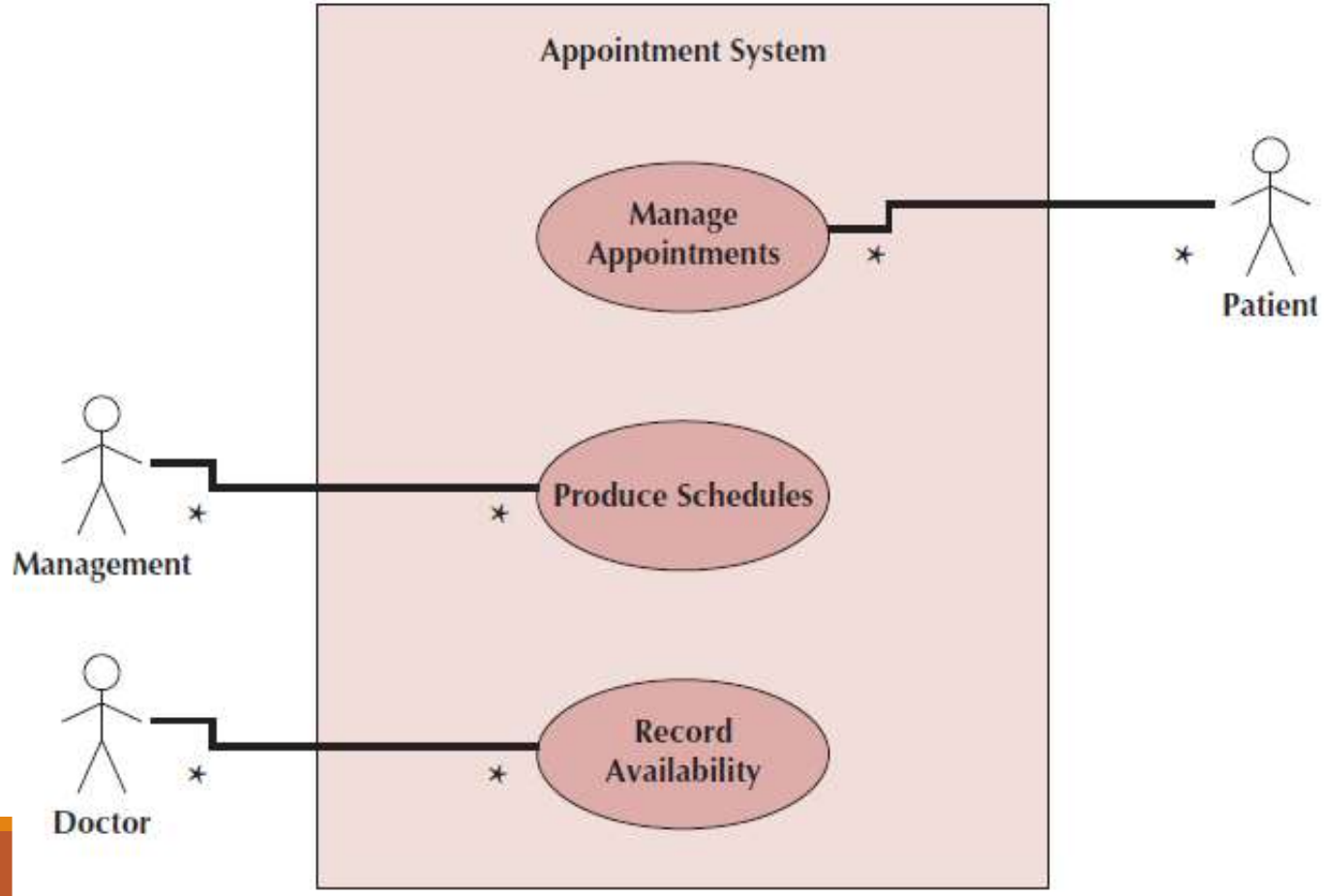
Sometimes an actor plays a specialized role of a more general type of actor. For example, there may be times when a new patient interacts with the system in a way that is somewhat different from a general patient. In this case, a *specialized actor* (i.e., new patient) can be placed on the model, shown using a line with a hollow triangle at the end of the more general actor (i.e., patient). The specialized actor inherits the behavior of the more general actor and extends it in some way.


Identity a "thing" from your requirements
Is the "thing" an actual person interacting with the system?
Yes
No
Is the "thing" something that I can change within the system's design?
No
Yes
The "thing" is *probably not* an actor. Anything that you can affect and have some control over when designing your system is likely to be considered a part of your system.
The "thing" is *probably* an actor. Be careful when it comes to people; some people can be considered part of your system.

A use case: ■ Represents a major piece of system functionality. ■ Can extend another use case. ■ Can include another use case. ■ Is placed inside the system boundary. ■ Is labeled with a descriptive verb–noun phrase.	
A subject boundary: ■ Includes the name of the subject inside or on top. ■ Represents the scope of the subject, e.g., a system or an individual business process.	
An association relationship: ■ Links an actor with the use case(s) with which it interacts.	

Use Case

A use case, depicted by an oval in the UML, is a major process that the system performs and that benefits an actor or actors in some way; it is labeled using a descriptive verb–noun phrase. We can tell from figure that the system has three primary use cases: Manage Appointments, Produce Schedule, and Record Availability.

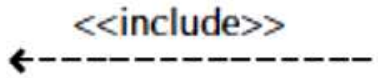
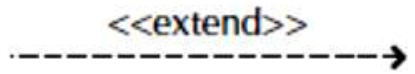


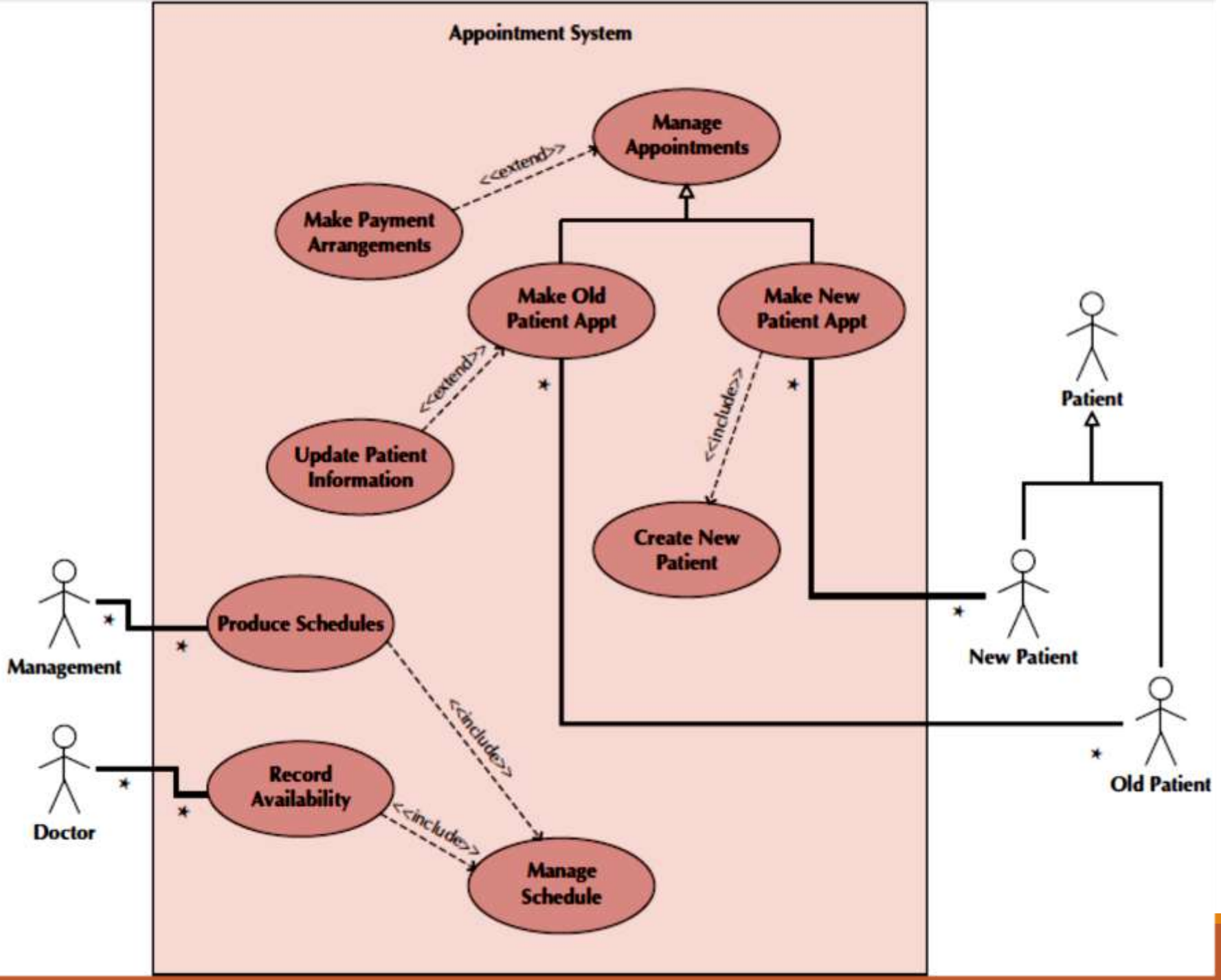
Association

Use cases are connected to actors through association relationships; these relationships show with which use cases the actors interact. A line drawn from an actor to a use case depicts an association. The association typically represents two way communication between the use case and the actor. If the communication is only one way, then a solid arrowhead can be used to designate the direction of the flow of information.

Subject Boundary

The use cases are enclosed within a *subject boundary,* which is a box that defines the scope of the system and clearly delineates what parts of the diagram are external or internal to it. One of the more difficult decisions to make is where to draw the subject boundary. A subject boundary can be used to separate a software system from its environment, a subsystem from other subsystems within the software system, or an individual process in a software system. They also can be used to separate an information system, including both software and internal actors, from its environment. Care should be taken to decide what the scope of the information system is to be.

An include relationship: ■ Represents the inclusion of the functionality of one use case within another. ■ Has an arrow drawn from the base use case to the used use case.	
An extend relationship: ■ Represents the extension of the use case to include optional behavior. ■ Has an arrow drawn from the extension use case to the base use case.	
A generalization relationship: ■ Represents a specialized use case to a more generalized one. ■ Has an arrow drawn from the specialized use case to the base use case.	

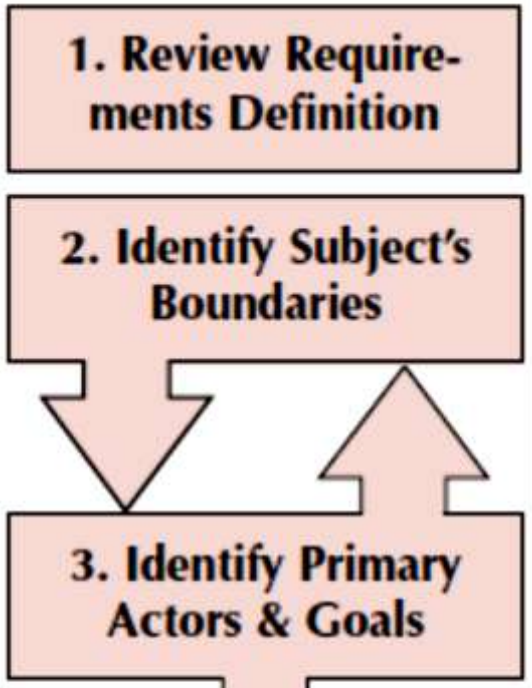

Appointment System
Manage Appointments
Make Payment Arrangements
<<extend>>
Make Old Patient Appt
Make New Patient Appt
<<extend>>
<<include>>
Update Patient Information
Create New Patient
Patient
Management
Produce Schedules
New Patient
<<include>>
Doctor
Record Availability
<<include>>
Manage Schedule
Old Patient

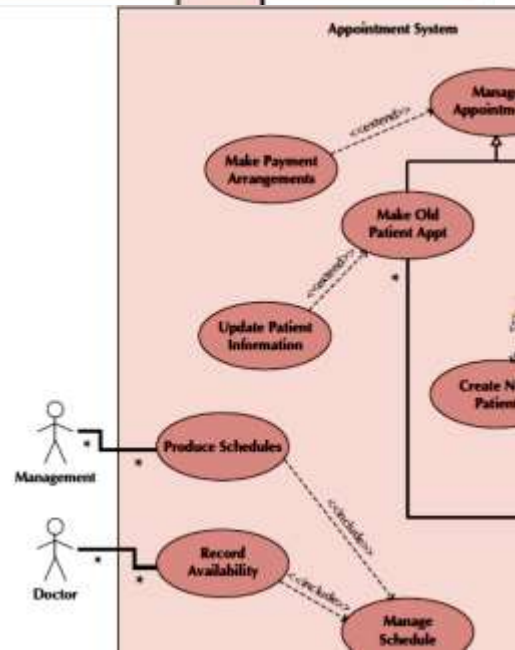
Identifying the Major Use Cases

The first step is to review the requirements definition. This helps the analyst to get a complete overview of the underlying business process being modeled.

The second step is to identify the subject's boundaries. This helps the analyst to identify the scope of the system. However, as we work through the development process, the boundary of the system most likely will change.

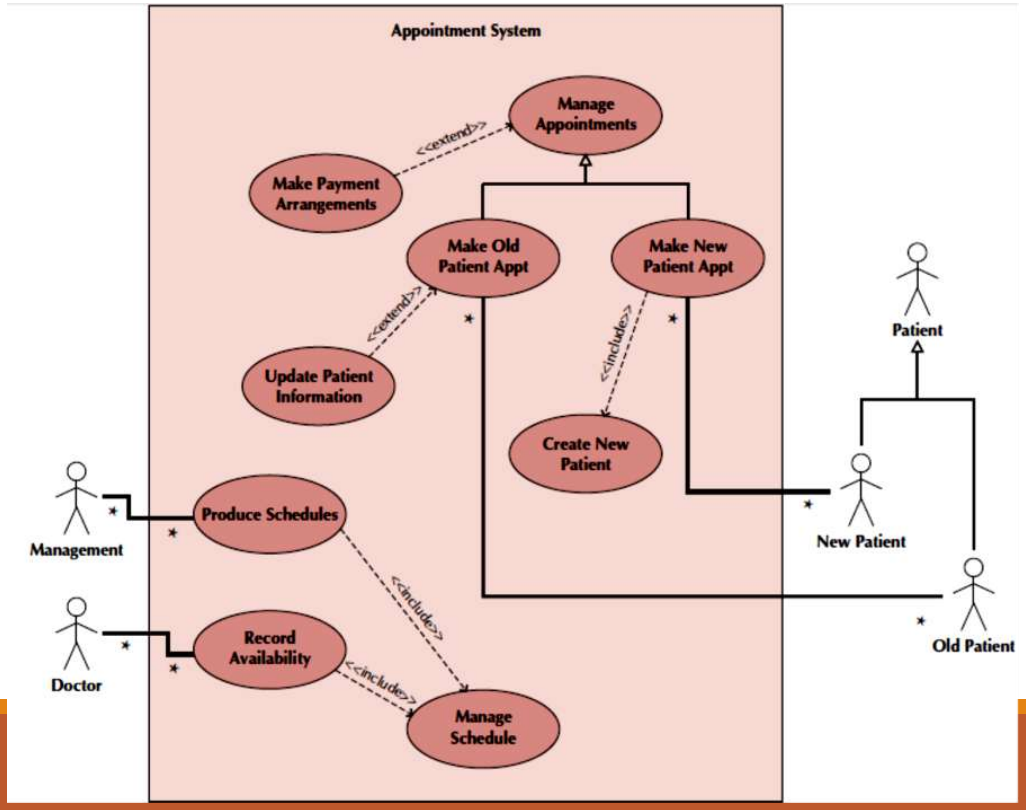
The third step is to identify the primary actors and their goals. The primary actors involved with the system comes from a list of stakeholders and users.

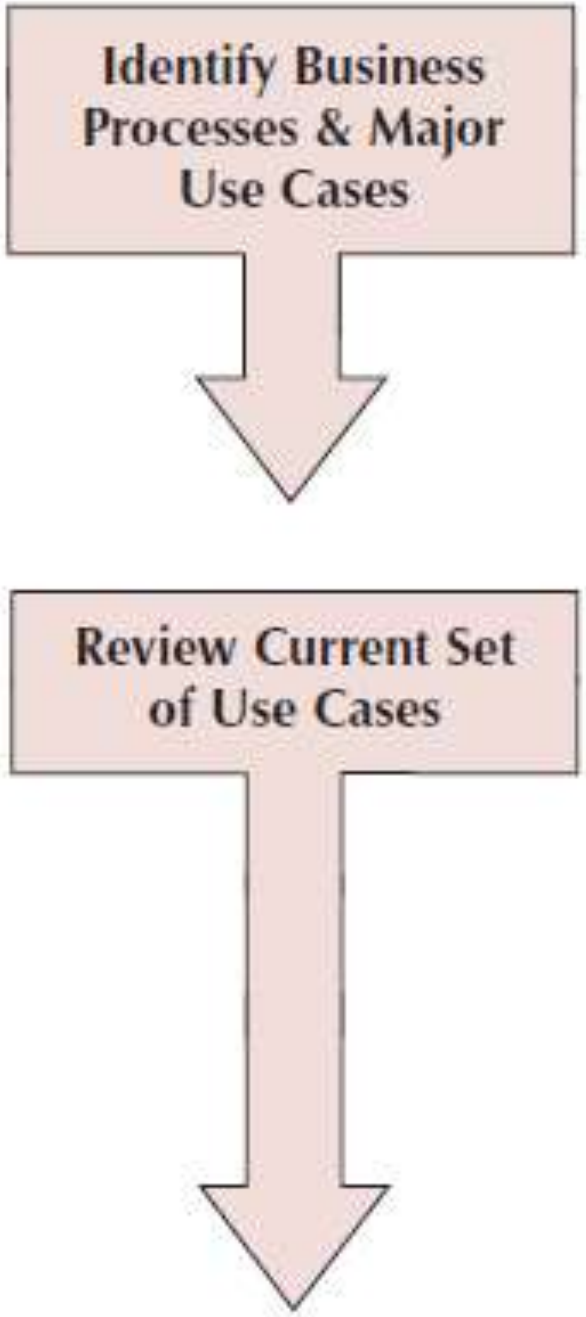




The fourth step is to simply identify the business processes and major use cases.

The fifth step is to carefully review the current set of use cases. It may be necessary to split some of them into multiple use cases or merge some of them into a single use case.





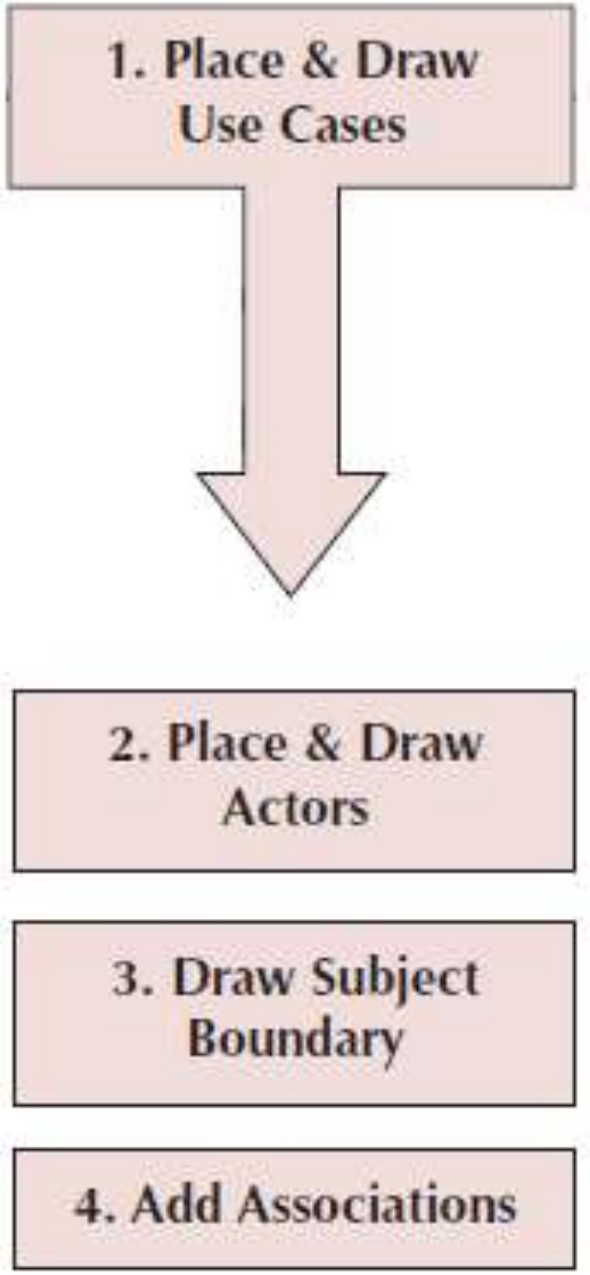
Creating a Use-Case Diagram

First, we place and draw the use cases on the diagram. These are taken directly from the major use cases previously identified. Special use-case associations (include, extend, or generalization) are also added to the model at this point.

Second, the actors are placed and drawn on the diagram. Like use-case placement, to minimize the number of lines that cross on the diagram, the actors should be placed near the use cases with which they are associated.

Third, the subject boundary is drawn. This forms the border of the subject, separating use cases (i.e., the subject's functionality) from actors (i.e., the roles of the external users).

The fourth and last step is to add associations by drawing lines to connect the actors to the use cases with which they interact. No order is implied by the diagram, and the items added along the way do not have to be placed in a particular order; therefore, it might help to rearrange the symbols a bit to minimize the number of lines that cross, making the diagram less confusing.



Example

The functional requirements for an automated university library circulation system include the need to support

- Searching
- Borrowing
- Book-maintenance activities.

The system should support searching

- by title
- by author
- by keywords
- by ISBN

Searching the library's collection database should be available on terminals in the library and available to potential borrowers via the Web. If the book of interest is currently checked out, a valid borrower should be allowed to request the book to be returned. Once the book has been checked back in, the borrower requesting the book should be notified of the book's availability.

The borrowing activities are built around checking books out and returning books by borrowers. There are three types of borrowers: **students, faculty or staff, and guests.** Regardless of the type of borrower, the borrower must have a valid ID card.

- If the borrower is a student, having the system check with the registrar's student database validates the ID card.
- If the borrower is a faculty or staff member, having the system check with the personnel office's employee database validates the ID card.
- If the borrower is a guest, the ID card is checked against the library's own borrower database.

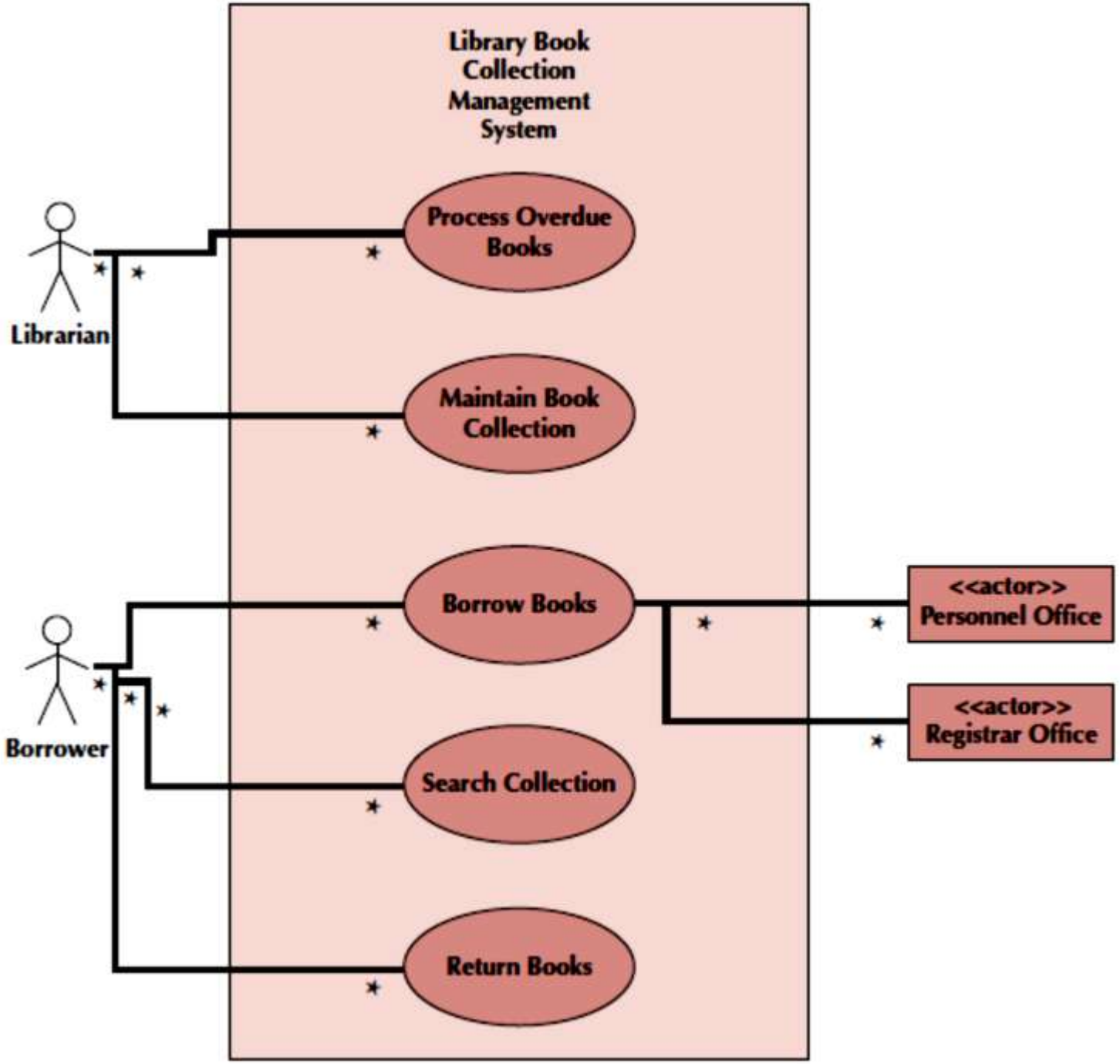
If the ID card is valid, the system must also check to determine whether the borrower has any overdue books or unpaid fines. If the ID card is invalid, the borrower has overdue books, or the borrower has unpaid fines, the system must reject the borrower's request to check out a book, otherwise the borrower's request should be honored. If a book is checked out, the system must update the library's collection database to reflect the book's new status.

The book-maintenance activities deal with adding and removing books from the library's book collection.

This requires a library manager to both logically and physically add and remove the book. Books being purchased by the library or books being returned in a damaged state typically cause these activities.

If a book is determined to be damaged when it is returned and it needs to be removed from the collection, the last borrower will be assessed a fine. However, if the book can be repaired, depending on the cost of the repair, the borrower might not be assessed a fine.

Every Monday, the library sends reminder emails to borrowers who have overdue books. If a book is overdue more than two weeks, the borrower is assessed a fine. Depending on how long the book remains overdue, the borrower can be assessed additional fines every Monday


Library Book Collection Management System
Process Overdue Books
Maintain Book Collection
Borrow Books
Search Collection
Return Books
Librarian
Borrower
<<actor>> Personnel Office
<<actor>> Registrar Office

Capturing a System Requirement

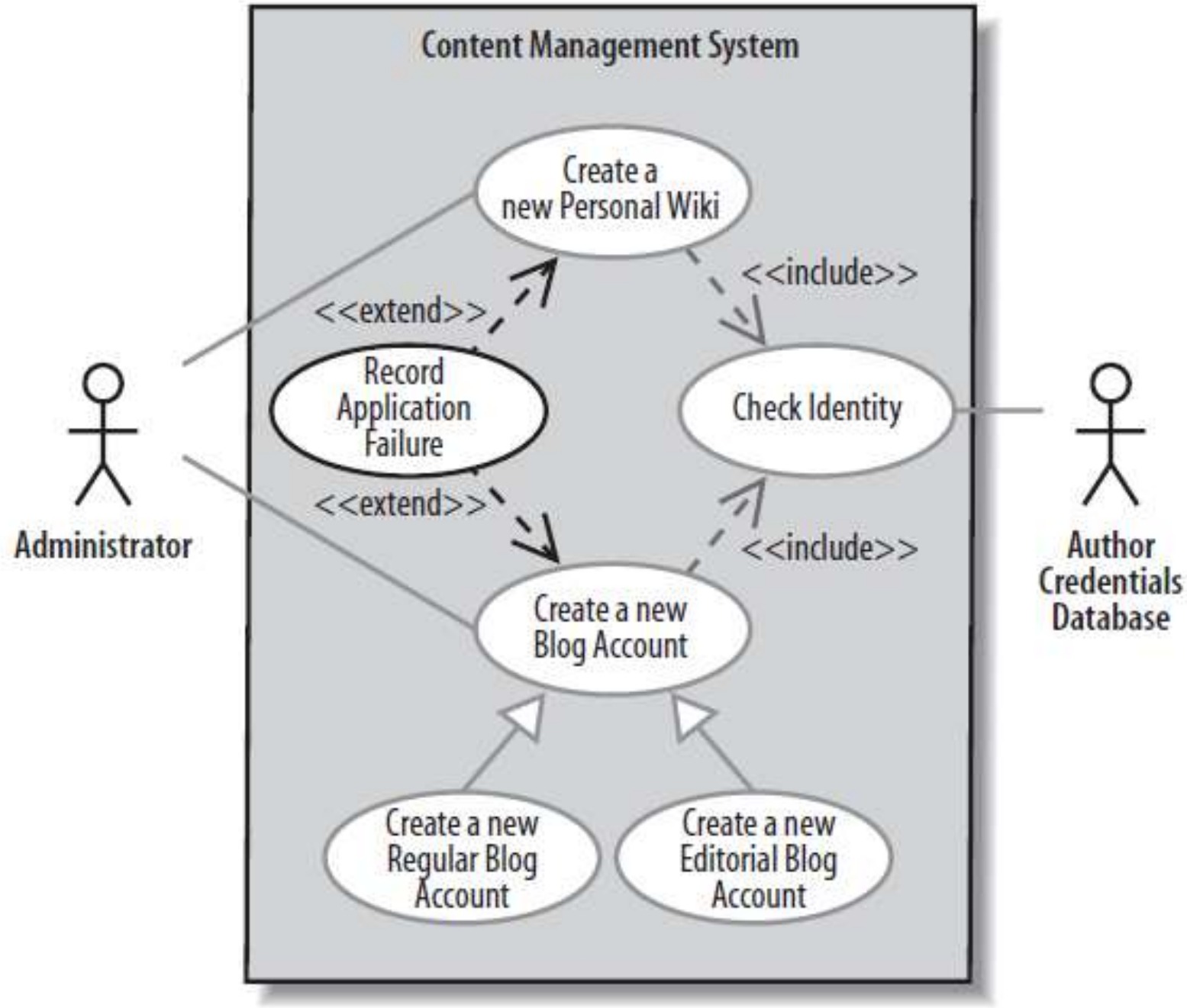
let’s take a look at a simple example. Suppose we’re defining requirements for a weblog content management system (CMS).

Requirement A.1

The content management system shall allow an administrator to create a new blog account, provided the personal details of the new blogger are verified using the author credentials database.

Requirement A.2

The content management system shall allow an administrator to create a new personal Wiki, provided the personal details of the applying author are verified using the Author Credentials Database.


Content Management System
Create a new Personal Wiki
<<include>>
<<extend>>
Record Application Failure
Check Identity
Administrator
<<extend>>
<<include>>
Author Credentials Database
Create a new Blog Account
Create a new Regular Blog Account
Create a new Editorial Blog Account

Business Analyst: The new flashcard function is needed so that students can remember the selected words. We believe that this function will improve the quality of the educational process and increase the results on the final tests. Also, a similar function is already available from competitors and we do not want to lose our competitive advantage.

The student adds a word-card to study from his dictionary. Then he opens the cards. For example, he sees a word in English or Russian, but does not see the translation. The student must remember the translation, then he can choose another card.

Teacher: The main thing is not only to learn the material, but also to consolidate it. I think the cards will help us. This is a kind of test. It is important that after the student remembered the translation, he can check himself. For example, he clicks on the card, sees the translation and chooses whether he remembered it correctly or not, and only after that we show him the next card. In the card details, you can see statistics so that the student can see his progress. It should also be possible to delete cards that the student has already learned. Oh yeah! It is necessary to switch translation modes - from Russian to English and from English to Russian.

Student 1: The most important thing is convenience. I want everything to be intuitive and work quickly. And beautiful, of course. Long loading is very annoying. I really like the dictionary function in similar applications. Can you do something similar, but so that it would be possible to repeat words, you know what I mean?

Student 2: Sometimes I forget the words that we covered in previous lessons. Of course, I can find them and look them up. But with each lesson, there are more and more such words. Can we somehow mark words to remember? It would be great if there was an option to select several words at once. For example, after completing a lesson, I would mark all the new words to remember in my dictionary.

BUSINESS PROCESS MODELING WITH ACTIVITY DIAGRAMS

Business process models describe the different activities that, when combined, support a business process. Business processes typically cut across functional departments (e.g., the creation of a new product involves many different activities that combine the efforts of many employees in many departments). From an object-oriented perspective, they cut across multiple objects. Many of the earlier object-oriented systems development approaches tended to ignore business process modeling.

The one **potential problem** of building business process models, from an object-oriented systems development perspective, is that they tend to reinforce a functional decomposition mindset. However, as long as they are used properly, business process models are very powerful tools for communicating the analyst's current understanding of the requirements to the user.

Martin Schedlbauer (*The Art of Business Process Modeling: The Business Analysts Guide to Process Modeling with UML*) provides a set of best practices to follow when modeling business processes.

- Be realistic, because it is virtually impossible to identify everything that is included in a business process at this point in the evolution of the system. Even if we could identify everything, everything is not equally important.
- Be agile because even though we might not identify every single feature of a business process, the features that we do identify should be identified in a rigorous manner.
- All modeling is a collaborative/social activity. Therefore, business process modeling must be performed with teams, not by individuals. When an individual creates a model, the chance of mixing up or omitting important tasks is greatly increased.

- Do not use a CASE tool to do the modeling but use whiteboards instead. However, once the process is understood, it is a good idea to use a CASE tool to document the process.
- Process modeling should be done in an iterative manner. In other words, as you better understand a business process, you will need to return to the documented version of the process and revise it. Remember, object-oriented system development is an iterative (and incremental) process.
- When modeling a business process, stay focused on that specific process. If tasks associated with other business processes are identified, simply record them on a to-do list and get back to the business process that you are currently modeling.

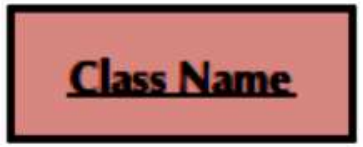
- Remember that a business process model is an abstraction of reality. By that, we mean that you should not include every minor task in the current description of the business process. Remember, you cannot afford to lose sight of the proverbial forest for the sake of detailed understanding of a single tree. Too many details at this point in the evolution of the system can cause confusion and actually prevent you from solving the underlying problem being addressed by the new system.

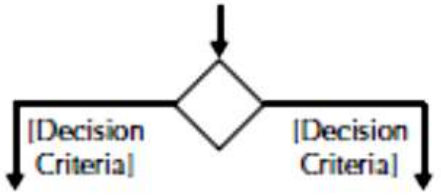
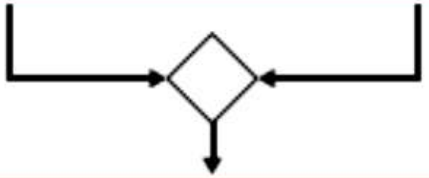
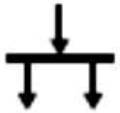
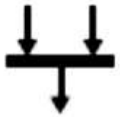
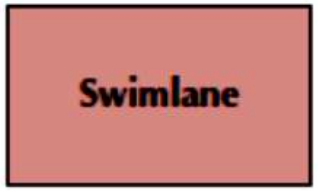
Activity diagram

An *activity diagram* can be used for any type of process-modeling activity. We describe their use in the context of business process modeling. *Process models* depict how a business system operates. They illustrate the processes or activities that are performed and how objects (data) move among them. A process model can be used to document a current system (i.e., as-is system) or a new system being developed (i.e., tobe system), whether computerized or not.

Use cases show what your system should do. Activity diagrams allow you to specify how your system will accomplish its goals. Activity diagrams show high-level actions chained together to represent a process occurring in your system.

Elements of an Activity Diagram

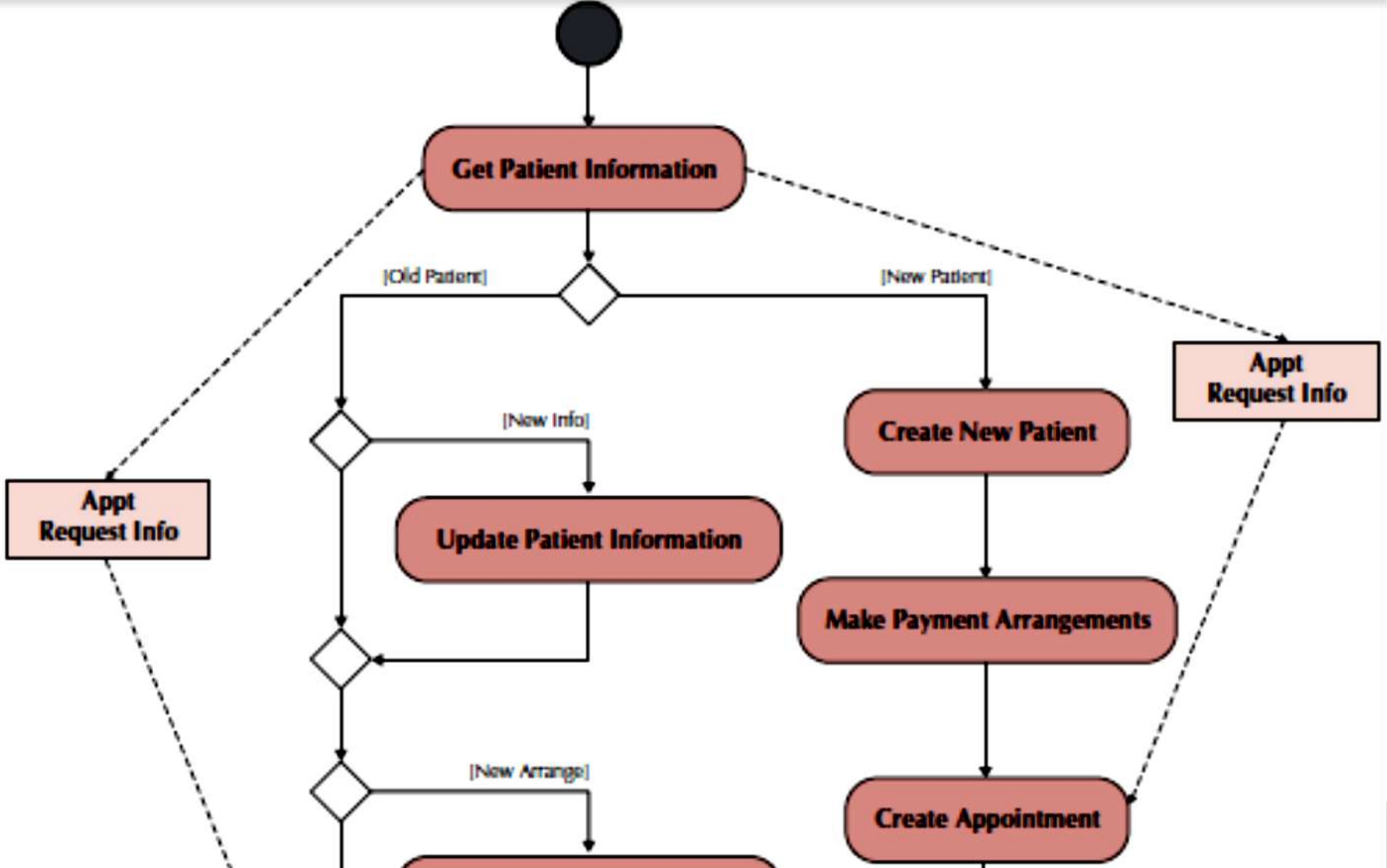
An action: ■ Is a simple, nondecomposable piece of behavior. ■ Is labeled by its name.	
An activity: ■ Is used to represent a set of actions. ■ Is labeled by its name.	
An object node: ■ Is used to represent an object that is connected to a set of object flows. ■ Is labeled by its class name.	
A control flow: ■ Shows the sequence of execution.	
An object flow: ■ Shows the flow of an object from one activity (or action) to another activity (or action).	
An initial node: ■ Portrays the beginning of a set of actions or activities.	

A final-activity node: ■ Is used to stop all control flows and object flows in an activity (or action).	
A final-flow node: ■ Is used to stop a specific control flow or object flow.	
A decision node: ■ Is used to represent a test condition to ensure that the control flow or object flow only goes down one path. ■ Is labeled with the decision criteria to continue down the specific path.	
A merge node: ■ Is used to bring back together different decision paths that were created using a decision node.	
A fork node: Is used to split behavior into a set of parallel or concurrent flows of activities (or actions)	
A join node: Is used to bring back together a set of parallel or concurrent flows of activities (or actions)	
A swimlane: Is used to break up an activity diagram into rows and columns to assign the individual activities (or actions) to the individuals or objects that are responsible for executing the activity (or action) Is labeled with the name of the individual or object responsible	

Actions and Activities

Actions* and *activities are performed for some specific business reason. Actions and activities can represent manual or computerized behavior. They are depicted in an activity diagram as a **rounded rectangle**. They should have a **name** that **begins with a verb** and **ends with a noun** (e.g., Get Patient Information or Make Payment Arrangements). Names should be short, yet contain enough information so that the reader can easily understand exactly what they do. The only **difference between an action and an activity is that an activity can be decomposed further into a set of activities and/or actions**, whereas an action represents a simple nondecomposable piece of the overall behavior being modeled. Typically, only activities are used for business process or workflow modeling.

In most cases, each activity is associated with a use case. The activity diagram in Figure shows a set of separate but related activities for the Manage Appointments use case: Get Patient Information, Update Patient Information, Create New Patient, Make Payment Arrangements, Make New Appointment, Change Appointment, and Cancel Appointment. Notice that the Make Payment Arrangements and Make New Appointment activities appear twice in the diagram; once for an “old” patient and once for a “new” patient.


Get Patient Information
[Old Patient]
[New Patient]
[New Info]
[New Arrange]
Update Patient Information
Create New Patient
Make Payment Arrangements
Create Appointment
Appt
Request Info
Appt
Request Info

Object Nodes

Activities and actions typically modify or transform objects. *Object nodes* model these objects in an activity diagram. Object nodes are portrayed in an activity diagram as rectangles. The name of the class of the object is written inside the rectangle.

Essentially, object nodes represent the flow of information from one activity to another activity. The simple appointment system portrayed in previous figure shows object nodes flowing from Get Patient Information activity.

Control Flows and Object Flows

There are two different types of flows in activity diagrams: control and object.

Control flows model the paths of execution through a business process. A control flow is portrayed as a solid line with an arrowhead on it showing the direction of flow. Control flows can be attached only to actions or activities. Previous figure portrays a set of control flows through the doctor's office's appointment system.

Object flows model the flow of objects through a business process. Because activities and actions modify or transform objects, object flows are necessary to show the actual objects that flow into and out of the actions or activities. An object flow is depicted as a dashed line with an arrowhead on it showing the direction of flow. An individual object flow must be attached to an action or activity on one end and an object node on the other end.

Control Nodes

There are seven different types of *control nodes* in an activity diagram:

- initial
- final-activity
- final-flow
- decision
- merge
- fork
- join

An ***initial node*** portrays the beginning of a set of actions or activities.12 An initial node is shown as a small filled-in circle.

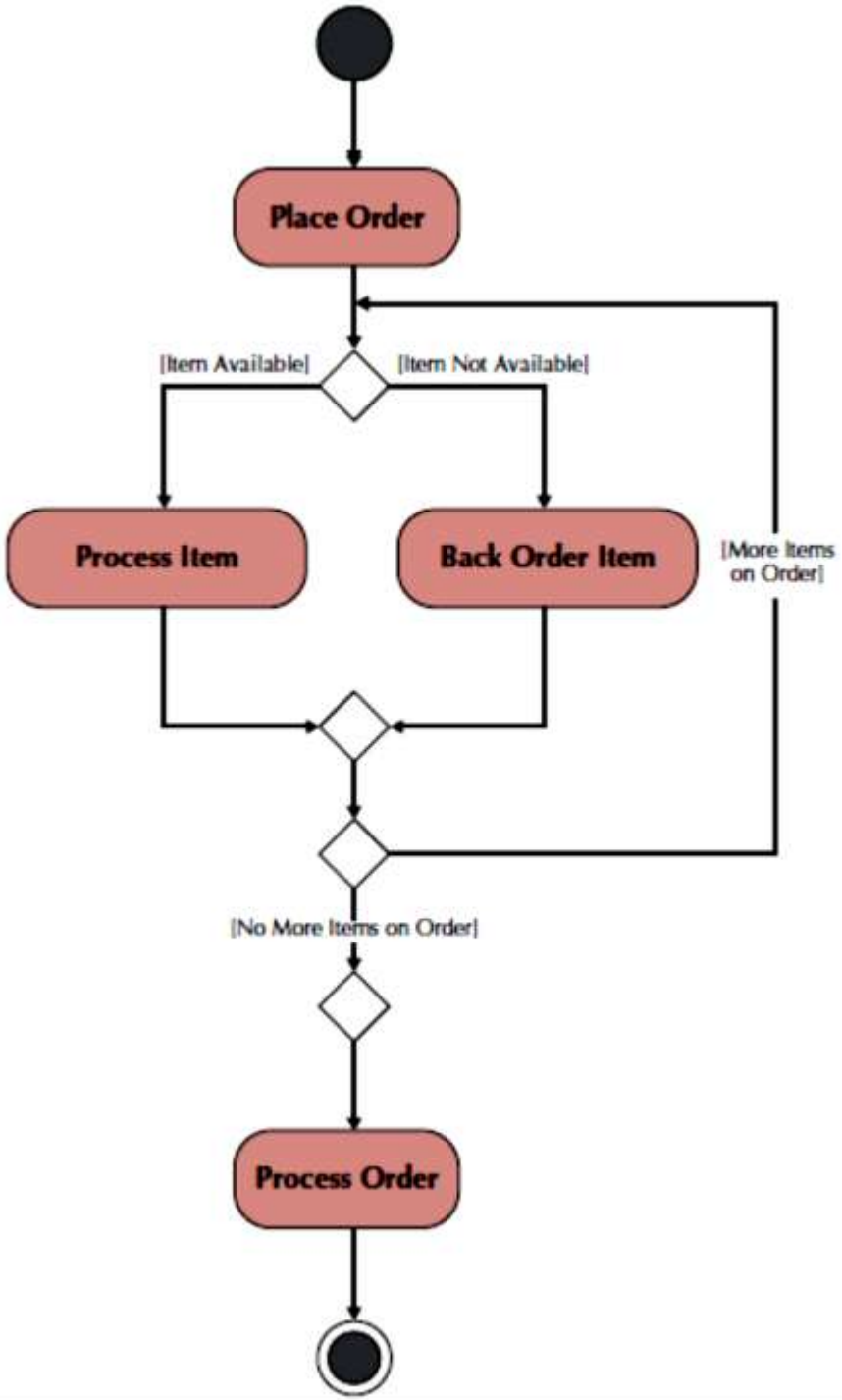
A ***final-activity node*** is used to stop the process being modeled. Any time a final-activity node is reached, all actions and activities are ended immediately, regardless of whether they are completed. A final-activity node is represented as a circle surrounding a small, filled-in circle, making it resemble a bull's-eye.

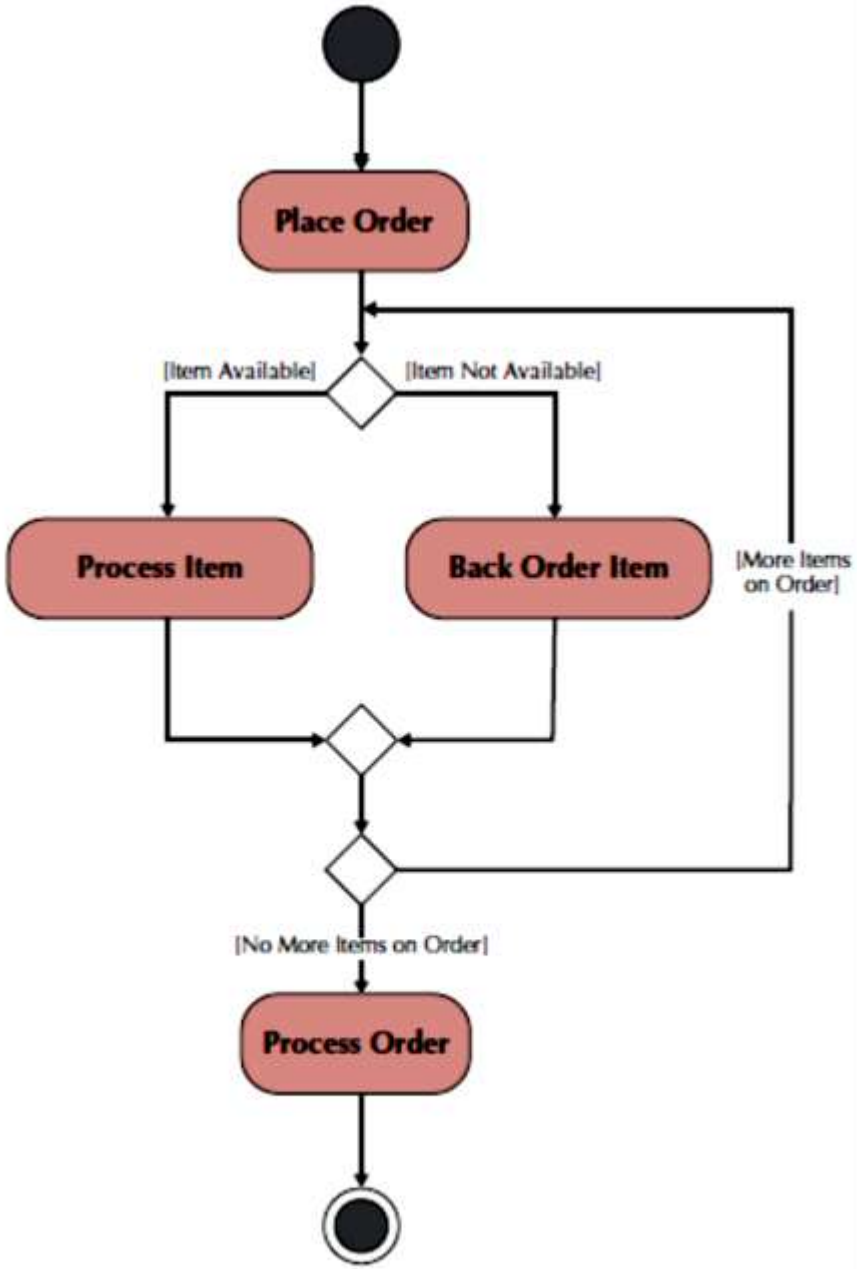
A ***final-flow node*** is similar to a final-activity node, except that it stops a specific path of execution through the business process but allows the other concurrent or parallel paths to continue. A final-flow node is shown as a small circle with an X in it.

The ***decision and merge nodes*** support modeling the decision structure of a business process. The ***decision node*** is used to represent the actual test condition that determines which of the paths exiting the decision node is to be traversed. In this case, each exiting path must be labeled with a guard condition. A *guard condition* represents the value of the test for that particular path to be executed.

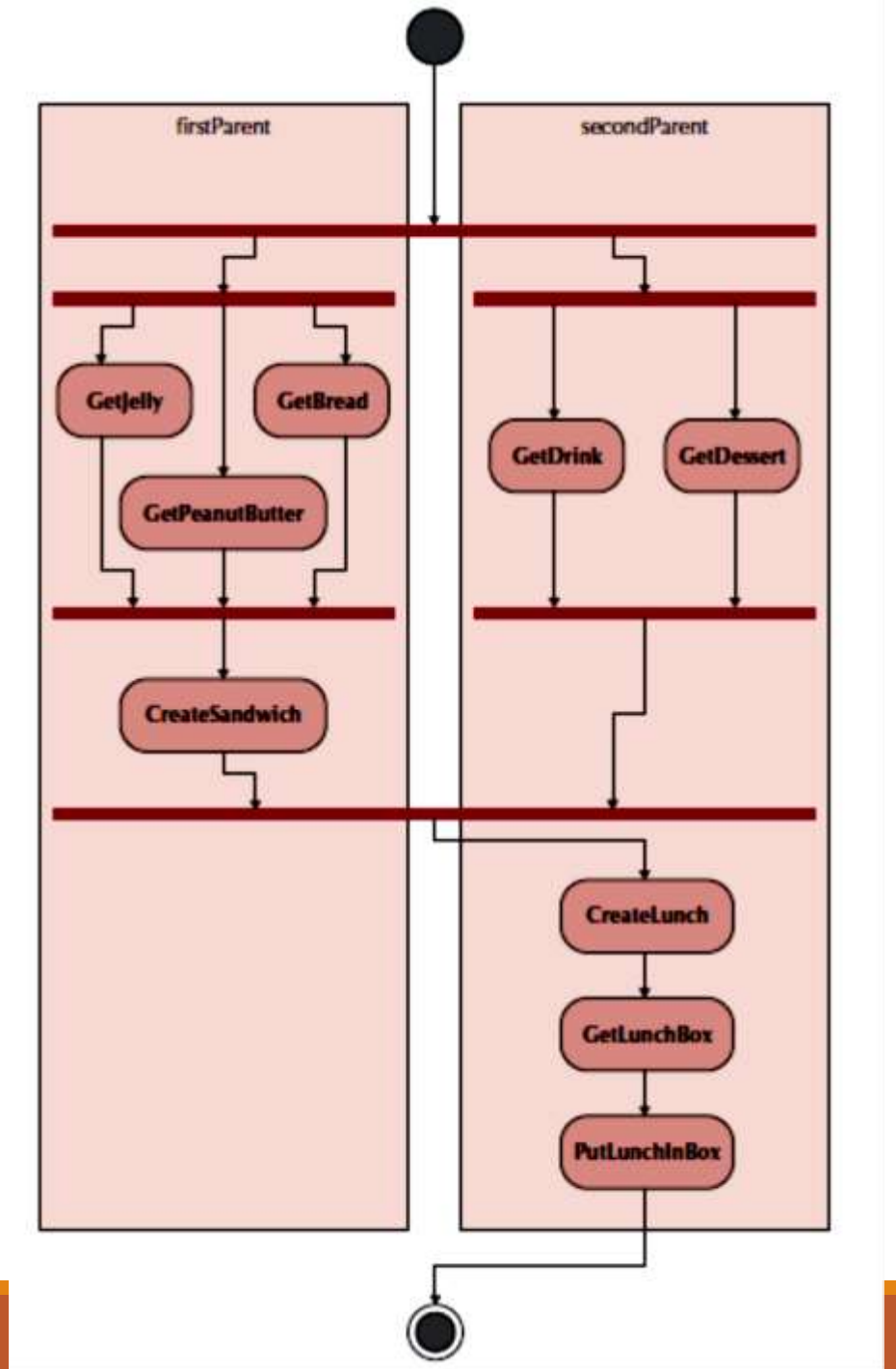
The *merge node* is used to bring back together multiple mutually exclusive paths that have been split based on an earlier decision However, sometimes, for clarity, it is better not to use a merge node. For example, in next figure, which of the two activity diagrams, both representing an overview level of an order process, is easier to understand, the one on the left or the one on the right?

The one on the left contains a merge node for the More Items on Order question, but the one on the right does not. In a sense, the decision node is playing double duty in the diagram on the right: It also serves as a merge node.


Place Order
[Item Available]
[Item Not Available]
Process Item
Back Order Item
[More Items on Order]
[No More Items on Order]
Process Order


Place Order
[Item Available]
[Item Not Available]
Process Item
Back Order Item
[More Items on Order]
[No More Items on Order]
Process Order

The fork and join nodes allow parallel and concurrent processes to be modeled. The ***fork node*** is used to split the behavior of the business process into multiple parallel or concurrent flows. Unlike the decision node, the paths are not mutually exclusive (i.e., both paths are executed concurrently). For example, in next figure, the fork node is used to show that two concurrent, parallel processes are to be executed. In this case, each process is executed by two separate processors (parents). The purpose of the join node is similar to that of the merge node. The ***join node*** simply brings back together the separate parallel or concurrent flows in the business process into a single flow.


firstParent
secondParent
GetJelly
GetBread
GetPeanutButter
CreateSandwich
GetDrink
GetDessert
CreateLunch
GetLunchBox
PutLunchInBox

Swimlanes

Activity diagrams can model a business process independent of any object implementation. However, there are times when it helps to break up an activity diagram in such a way that it can be used to assign responsibility to objects or individuals who would actually perform the activity. This is especially useful when modeling a business workflow and is accomplished through the use of *swimlanes.* In prev figure, the swimlanes are used to break up among two parents the making of a school lunch comprising a peanut butter and jelly sandwich, a drink, and dessert. In this case, we use vertical swimlanes. We could also draw the activity diagram using more of a left-to-right orientation instead of a top-down orientation. In that case, the swimlanes are drawn horizontally.

In an actual business workflow, there would be activities that should be associated with roles of individuals involved in the business workflow (e.g., employees or customers) and the activities to be accomplished by the information system being created.

Guidelines for Creating Activity Diagrams

Scott Ambler (*The Object Primer: The Application Developer's Guide to Object Orientation and the UML,* 2nd ed.) suggests the following guidelines when creating activity diagrams:

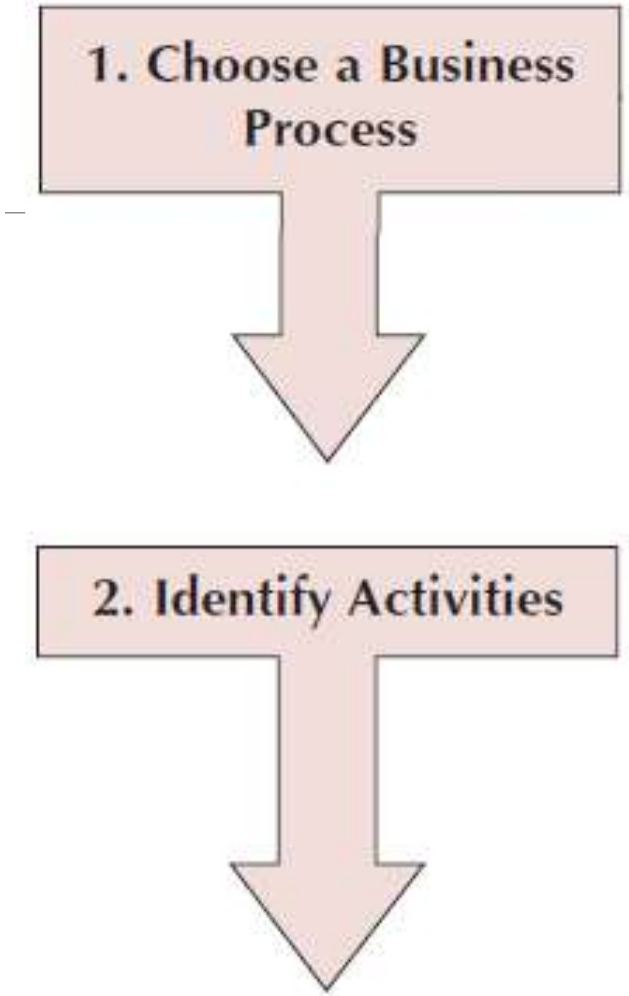
- Because an activity diagram can be used to model any kind of process, you should set the context or scope of the activity being modeled. Once you have determined the scope, you should give the diagram an appropriate title.
- You must identify the activities, control flows, and object flows that occur between the activities.
- You should identify any decisions that are part of the process being modeled.
- You should attempt to identify any prospects for parallelism in the process.
- You should draw the activity diagram.

Creating Activity Diagrams

There are five steps in creating an activity diagram to document and model a business process.

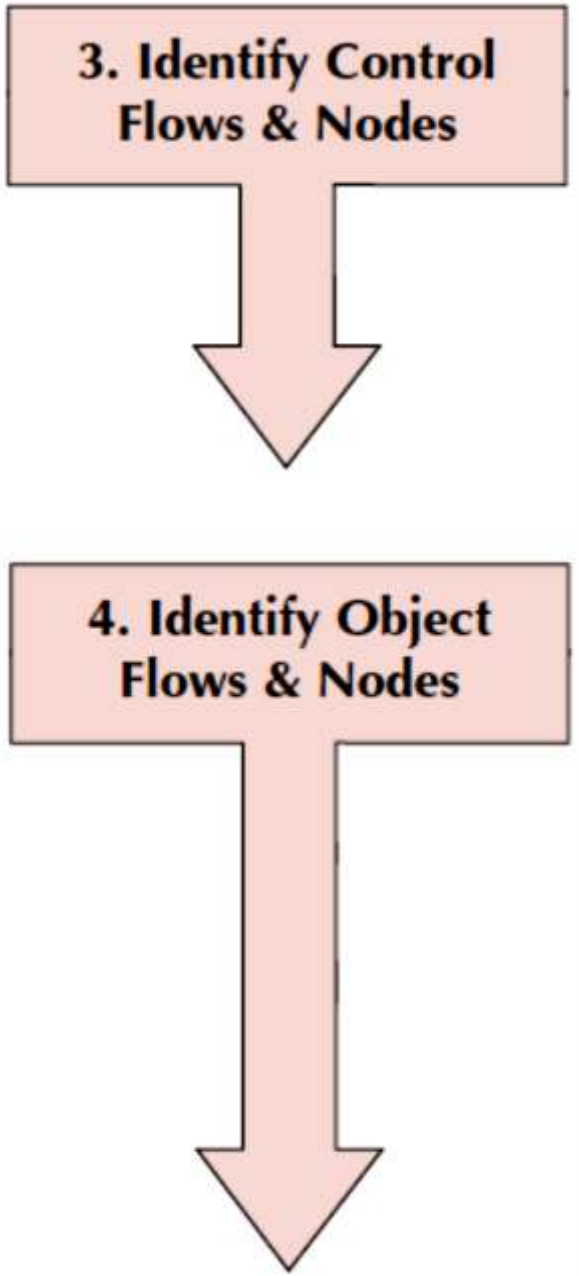
First, you must choose a business process that was previously identified to model. To do this, you should review the requirements definition and the use-case diagram created to represent the requirements. You should also review all of the documentation created during the requirements-gathering process. In most cases, the use cases on the use-case diagram will be the best place to start.

Second, identify the set of activities necessary to support the business process. Based on this information, we can identify a set of activities. In this case, the activities are Update Patient Information, Make Payment Arrangements, Create New Patient, Create Appointment, Cancel Appointment, and Change Appointment.

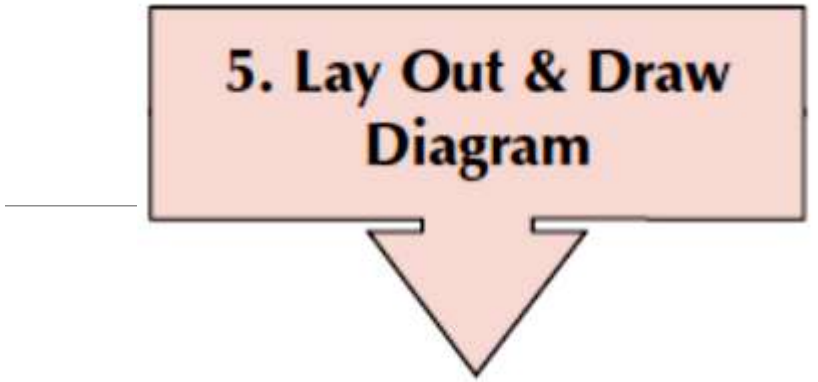


Third, identify the control flows and nodes necessary to document the logic of the business process.

Fourth, identify the object flows and nodes necessary to support the logic of the business process. Typically object nodes and flows are not shown on many activity diagrams used to model a business process. The primary exception is if information captured by the system in one activity is used in an activity that is performed later, but *not* immediately after the activity that captured the information. In the appointment example, it is obvious that we need to be able to determine whether the patient is an old or new patient and the type of action that the patient would like to have performed (create, cancel, or change an appointment).



Fifth, lay out and draw the activity diagram to document the business process. For esthetic and understandability reasons, just as when drawing a use-case diagram, you should attempt to minimize potential line crossings.



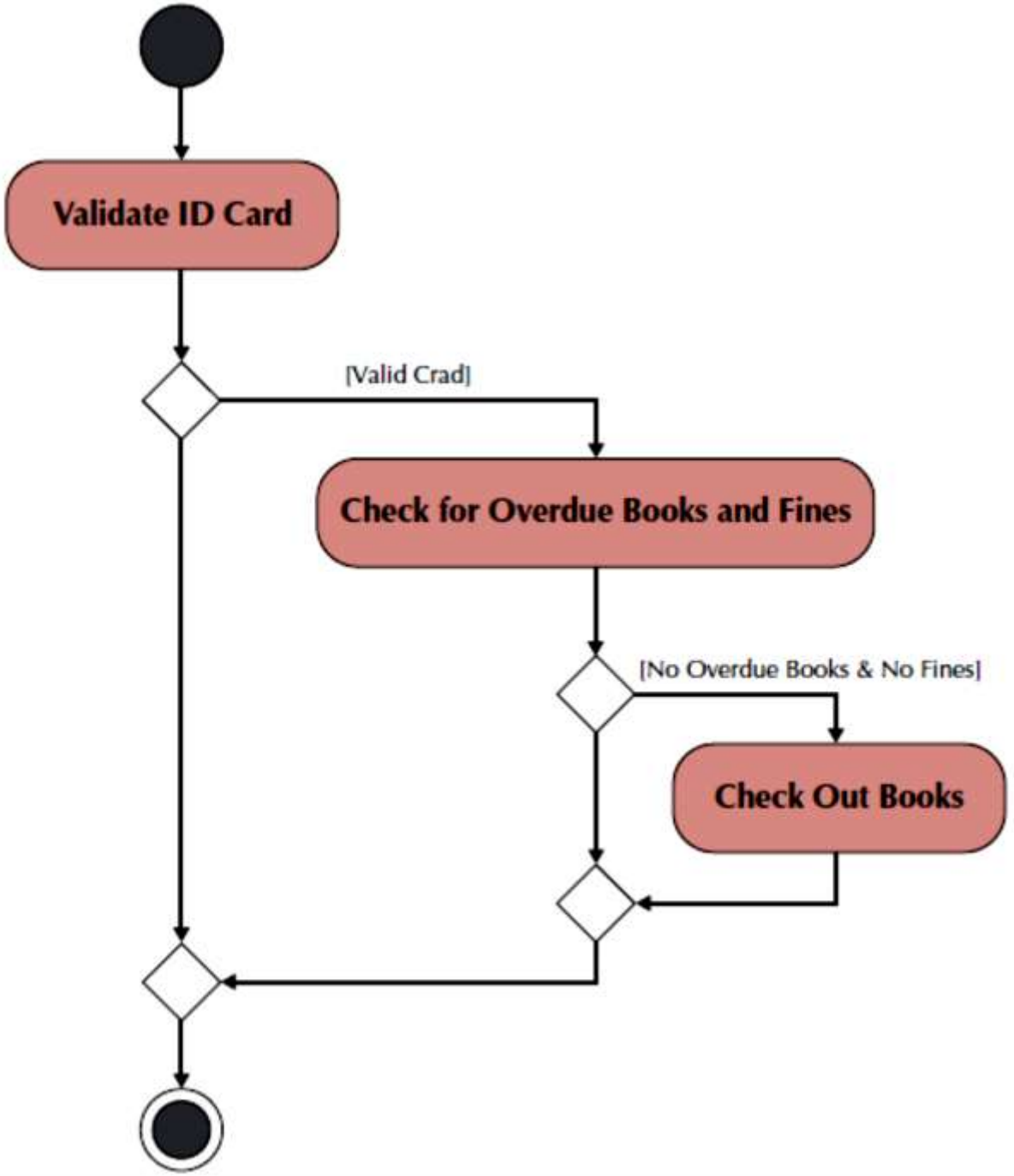
Example

The first step is to choose a business process to model. In this case, we want to create an activity diagram for the Borrow Books use case. The functional requirements for this use case were:

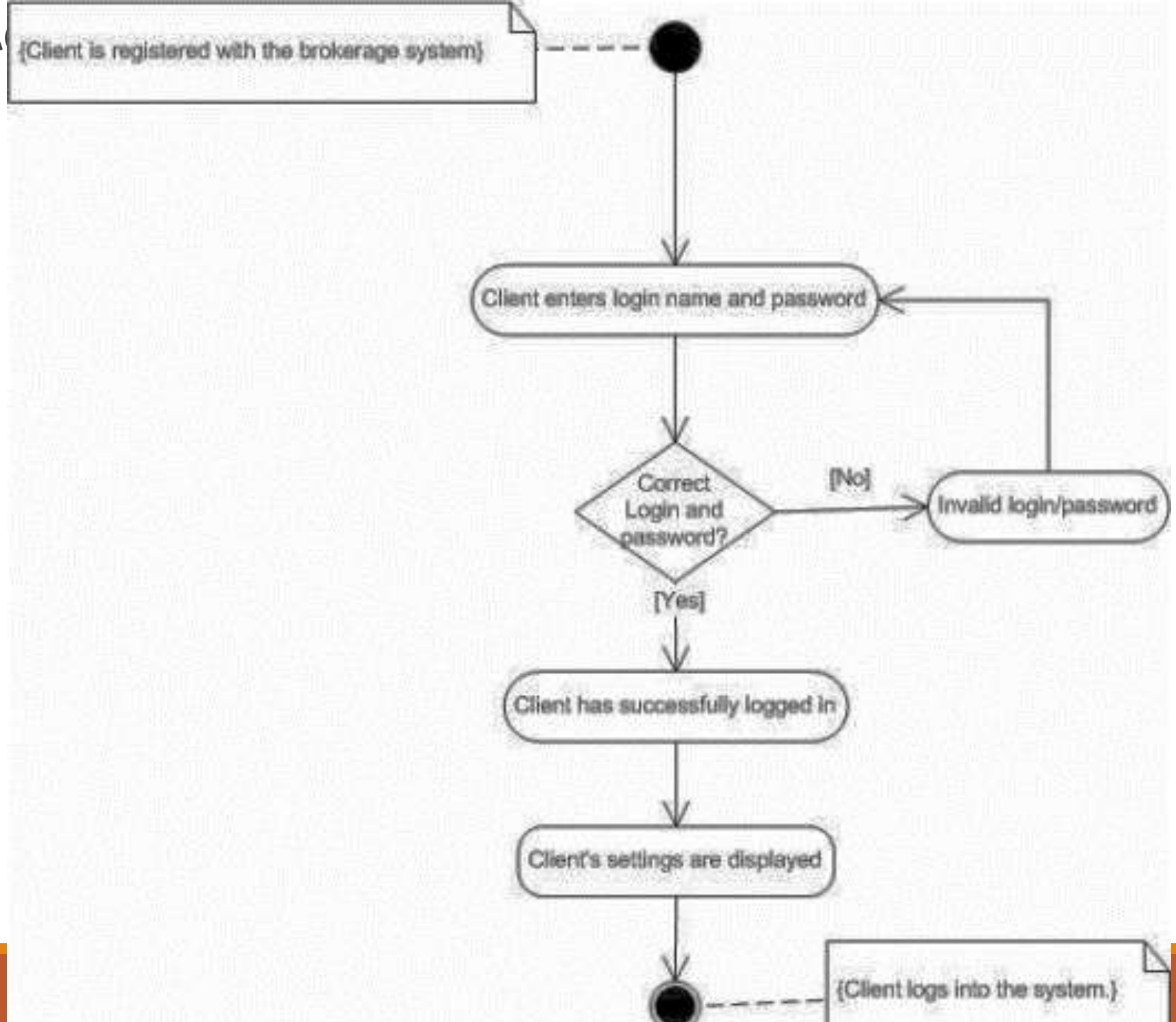
The borrowing activities are built around checking books out and returning books by borrowers. There are three types of borrowers: students, faculty or staff, and guests. Regardless of the type of borrower, the borrower must have a valid ID card. If the borrower is a student, having the system check with the registrar's student database validates the ID card. If the borrower is a faculty or staff member, having the system check with the personnel office's employee database validates the ID card.

If the borrower is a guest, the ID card is checked against the library's own borrower database. If the ID card is valid, the system must also check to determine whether the borrower has any overdue books or unpaid fines. If the ID card is invalid, the borrower has overdue books, or the borrower has unpaid fines, the system must reject the borrower's request to check out a book, otherwise the borrower's request should be honored.

The second step to model a business process is to identify the activities that make up the process. Based on the requirements for the Borrow Books use case, we can identify three major activities: Validate ID Card, Check for Overdue Books and Fines, and Check Out Books. The third step is to identify the control flows and control nodes necessary to model the decision logic of the business process. In this case there obviously will have to be an initial node, a final-flow node, and a set of decision and merge nodes for each decision to be made. The fourth step is to identify the object flows and object nodes necessary to complete the description of the business process. In this case, there really is no need to include object nodes and flows. Finally, we can lay out the diagram.


Validate ID Card
[Valid Crad]
Check for Overdue Books and Fines
[No Overdue Books & No Fines]
Check Out Books

Create an A


{Client is registered with the brokerage system}
Client enters login name and password
Correct Login and password?
[No]
Invalid login/password
[Yes]
Client has successfully logged in
Client's settings are displayed
{Client logs into the system.}

Example

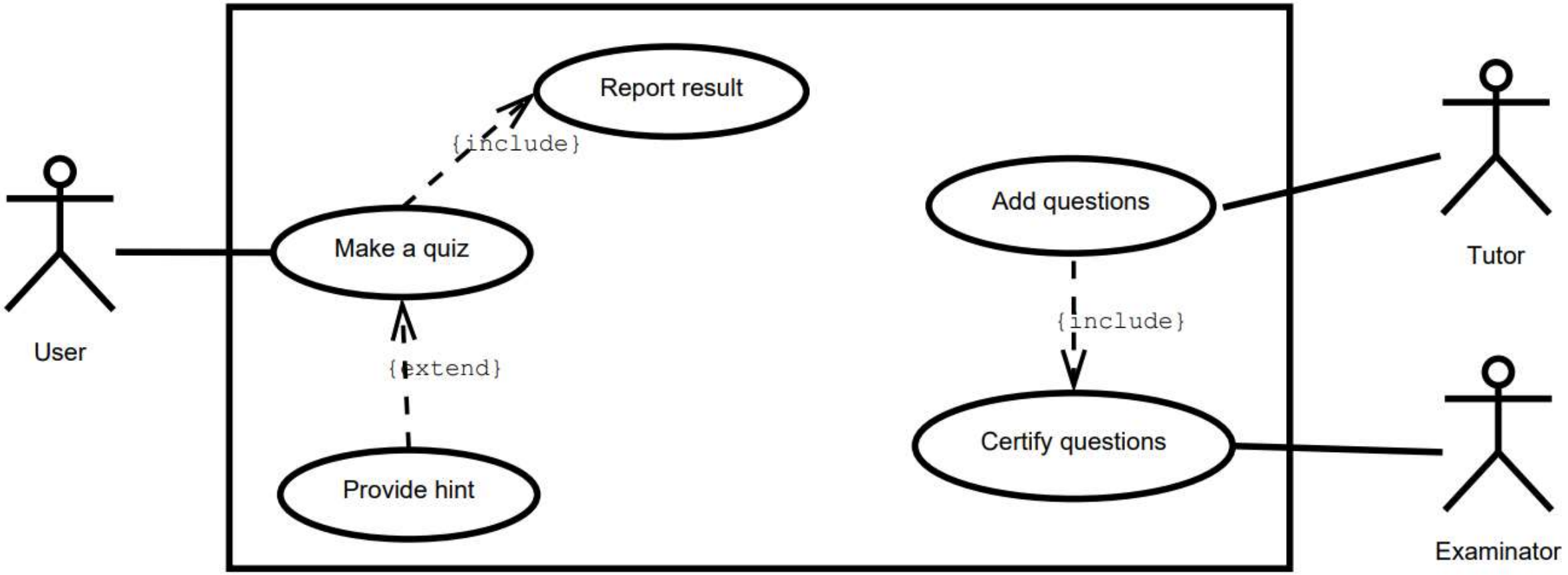
Do you know that it costs a lot of money to get a ’Certified Java Programmer’ certificate? It could cost you thousands of euros. Let’s imagine we will develop a browser-based training system to help people prepare for such a certification exam.

Requirements:

1.A user can request a quiz for the system. The system picks a set of questions from its database, and compose them together to make a quiz. It rates the user’s answers, and gives hints if the user requests it.

2. In addition to users, we also have tutors who provide questions and hints. And also examinators who must certify questions to make sure they are not too trivial, and that they are sensical.

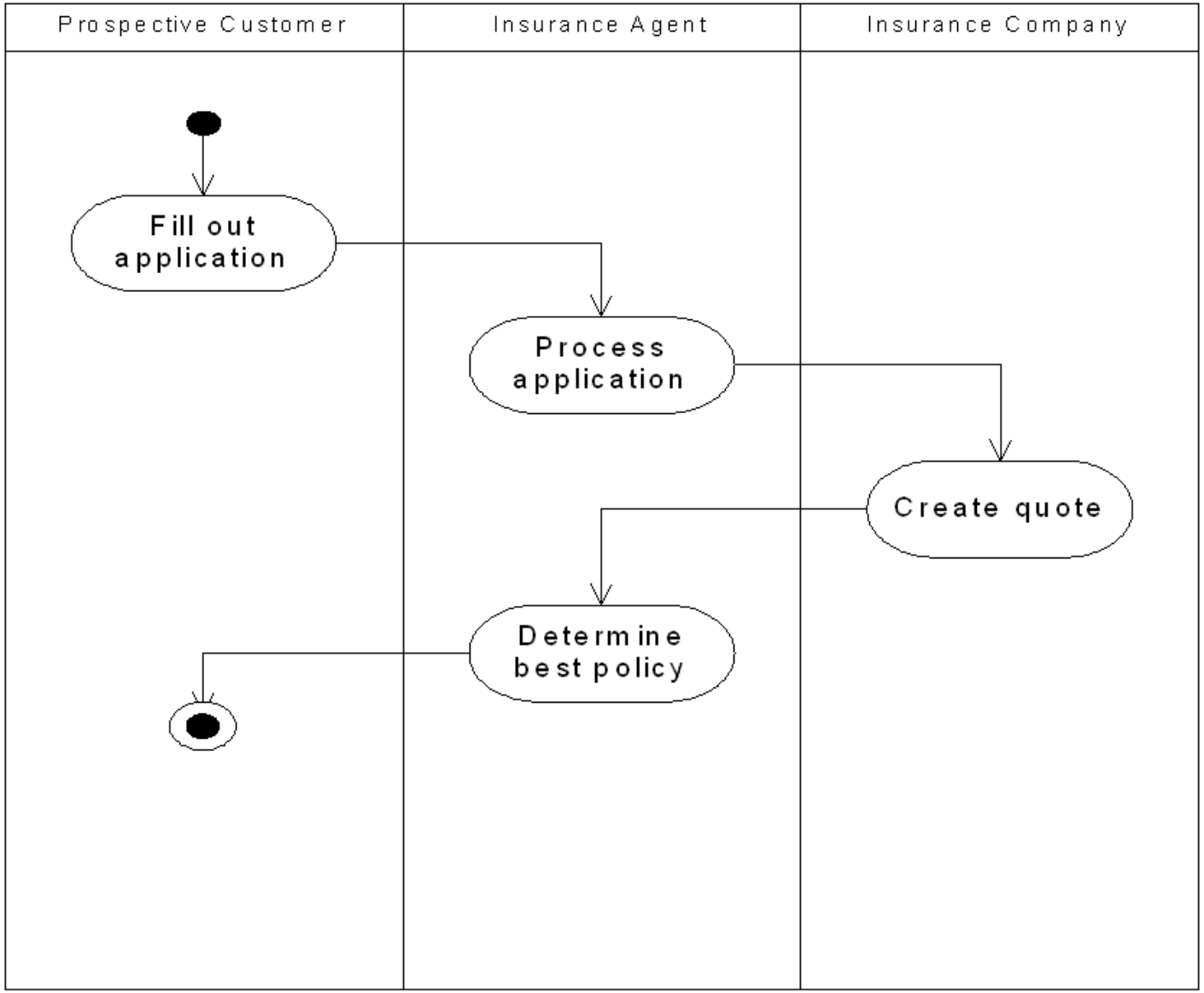
Make a use case diagram to model this system. Work out some of your use cases. Since we don’t have real stake holders here, you are free to fill in details you think is sensical for this example.


Report result
{include}
Make a quiz
{extend}
Provide hint
User
Add questions
{include}
Certify questions
Tutor
Examinator

Activity diagram

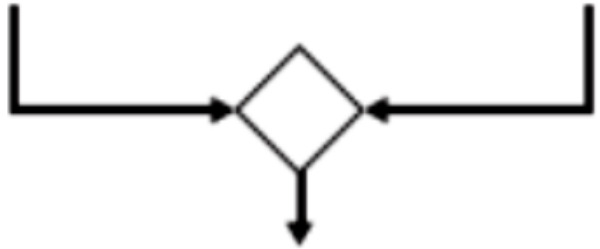
Develop an activity diagram based on the following narrative.

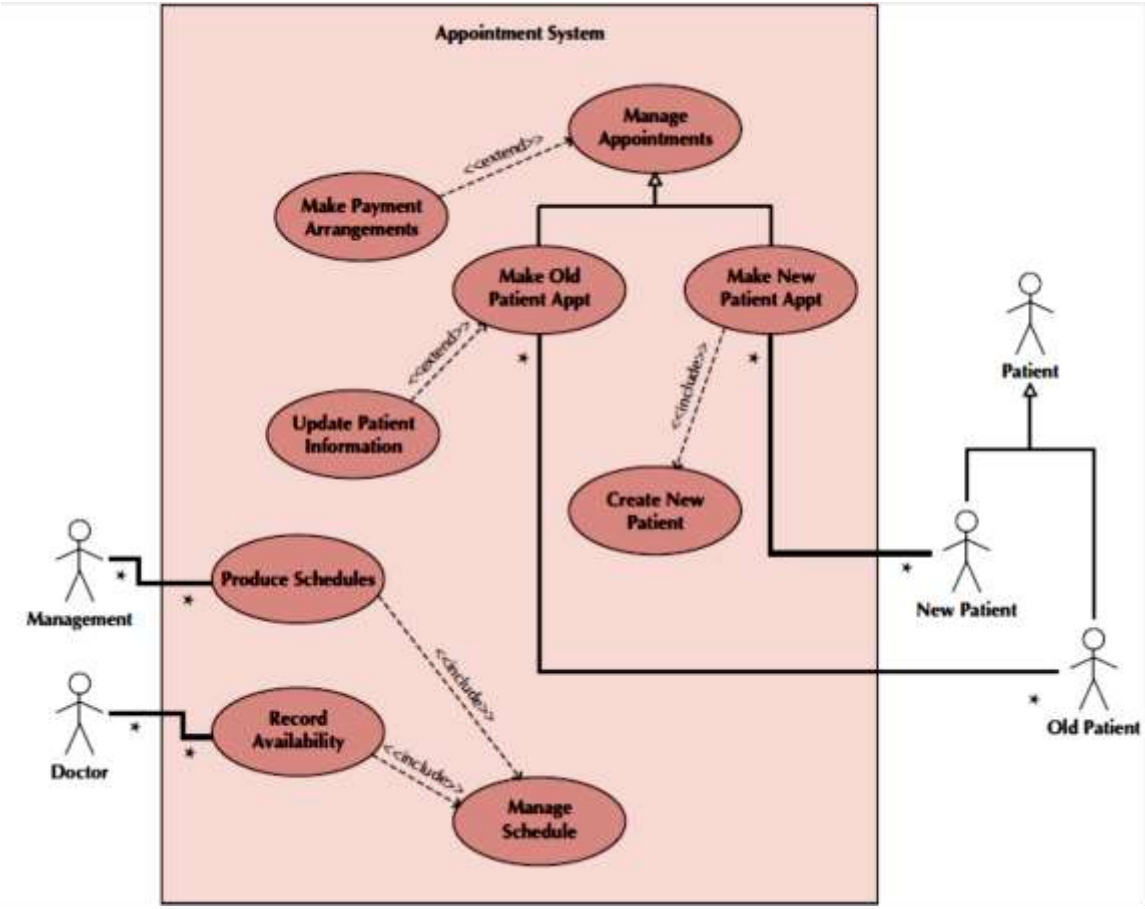
The purpose of the Open Access Insurance System is to provide automotive insurance to car owners. Initially, prospective customers fill out an insurance application, which provides information about the customer and his or her vehicles. This information is sent to an agent, who sends it to various insurance companies to get quotes for insurance. When the responses return, the agent then determines the best policy for the type and level of coverage desired and gives the customer a copy of the insurance policy proposal and quote.


Prospective Customer
Insurance Agent
Insurance Company
Fill out application
Process application
Create quote
Determine best policy

1. _____ diagrams are simply a graphical overview of the set of functions contained in the system. They illustrate the main functionality of a system and the actors that interact with the system.

2. The name of this element In Activity diagram _____.




Appointment System
Manage Appointments
<<extend>>
Make Payment Arrangements
Make Old Patient Appt
Make New Patient Appt
<<extend>>
<<include>>
Update Patient Information
Create New Patient
Patient
New Patient
Old Patient
Management
Produce Schedules
Doctor
Record Availability
<<include>>
<<include>>
Manage Schedule